

Non-small cell lung cancer (NSCLC) is histologically divided into adenocarcinoma, squamous cell carcinoma, and large-cell carcinoma. Lung adenocarcinoma (LUAD) is the most common type, accounting for approximately 40% of all lung cancers. LUAD is often detected at the metastatic stage with prevalence in the brain, bones, and respiratory system. Distant metastasis is the major cause of mortality in lung cancer; however, specific aspects of metastatic lung cancer and its associated microenvironments remain poorly understood.

Efforts made for the understanding of lung cancer progression and metastasis have largely focused on profiling of cancer cells with genetic aberrations. However, progression and metastasis are also influenced by complex and dynamic features in tumor surroundings.

To investigate stromal cell dynamics in the tumor microenvironment we will focus on endothelial cells. With the knowledge you already have in single cell analysis, determine de subclusters comprised in this endothelial population and if any of them are related to the tumor

▼ Import the libraries we will use to analyze the data

```
import sys
IN_COLAB = 'google.colab' in sys.modules

if IN_COLAB:
    !pip install scanpy
```

```
Requirement already satisfied: scanpy in /usr/local/lib/python3.10/dist-packages (1.10
Requirement already satisfied: anndata>=0.8 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: h5py>=3.1 in /usr/local/lib/python3.10/dist-packages (f
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: legacy-api-wrap>=1.4 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: matplotlib>=3.6 in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: natsort in /usr/local/lib/python3.10/dist-packages (fro
Requirement already satisfied: networkx>=2.7 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: numba>=0.56 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: pandas>=1.5 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: patsy in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: pynndescent>=0.5 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: scikit-learn>=0.24 in /usr/local/lib/python3.10/dist-pa
Requirement already satisfied: scipy>=1.8 in /usr/local/lib/python3.10/dist-packages (
Requirement already satisfied: seaborn>=0.13 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: session-info in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: statsmodels>=0.13 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from s
Requirement already satisfied: umap-learn!=0.5.0,>=0.5 in /usr/local/lib/python3.10/di
Requirement already satisfied: array-api-compat!=1.5,>1.4 in /usr/local/lib/python3.10
Requirement already satisfied: exceptiongroup in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-pac
```

```
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: llvmlite<0.42,>=0.41.0dev0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from packaging)
Requirement already satisfied: stdlib-list in /usr/local/lib/python3.10/dist-packages
```

```
import scanpy as sc
import numpy as np
import scipy as sp
import pandas as pd
import csv as csv
import seaborn as sb
```

```
import matplotlib.pyplot as plt
from matplotlib import rcParams
from matplotlib import colors
import gc
```

```
from scipy import sparse
import os
```

▼ Download and read the file with raw data

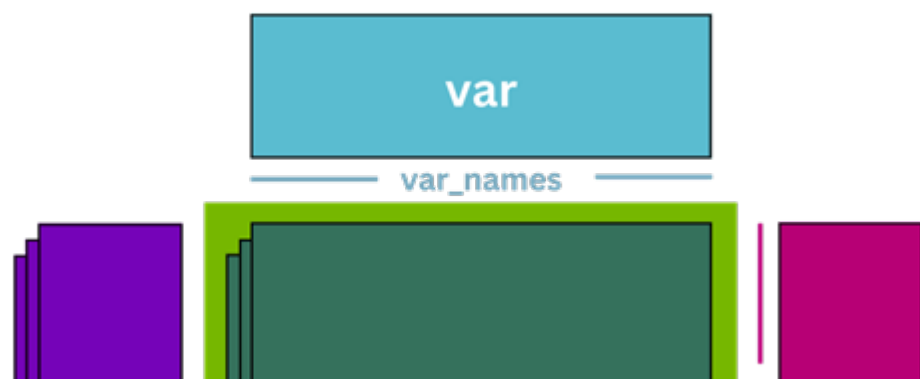
```
# Raw data
if not os.path.exists('endothelial_matrix_raw.h5ad'):
    !wget --no-check-certificate 'https://docs.google.com/uc?export=download&id=13xNEUa0jv'
```

```
adata = sc.read_h5ad('./endothelial_matrix_raw.h5ad')
```

```
adata.var_names
```

```
➡ Index(['A1BG', 'A1BG-AS1', 'A1CF', 'A2M', 'A2M-AS1', 'A2ML1', 'A2ML1-AS1',
        'A2ML1-AS2', 'A4GALT', 'A4GNT',
        ...,
        'ZXDA', 'ZXDB', 'ZXDC', 'ZYG11A', 'ZYG11B', 'ZYX', 'ZZEF1', 'ZZZ3',
        'bP-21264C1.2', 'bP-218909.3'],
        dtype='object', length=29634)
```

How the anndata object looks like:



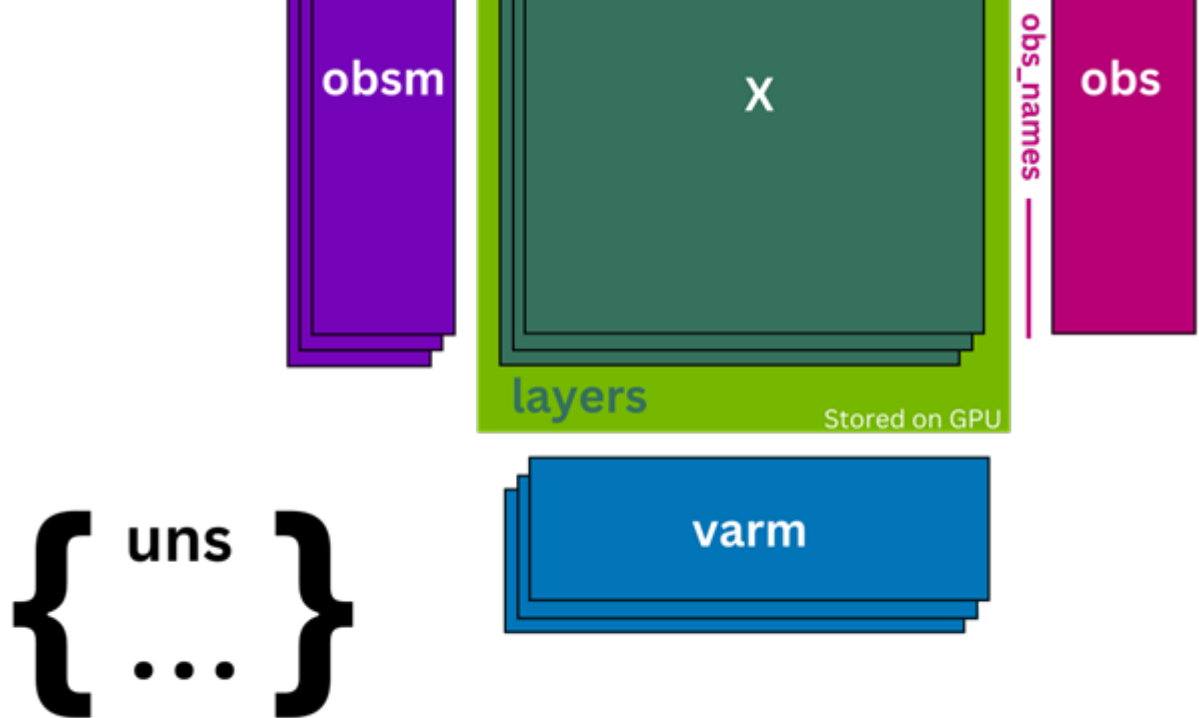


image.png

What information do we have in this anndata object?

```
adata
→ AnnData object with n_obs × n_vars = 1996 × 29634
  obs: 'Index', 'Barcode', 'Sample', 'Sample_Origin', 'Cell_type',
'Cell_type.refined', 'Cell_subtype'
  var: 'var_names'
```

```
adata.obs['Sample_Origin']
→ AGTCTTTGTGACTCAT_LUNG_N01      nLung
   ATCACGACAAGCGCTC_LUNG_N01      nLung
   CACAAACAGAAGGCCT_LUNG_N01      nLung
   CAGGTGCAGAGACTAT_LUNG_N01      nLung
   CATTCGCAGGCATGTG_LUNG_N01      nLung
   ...
   TATCAGGGTCAAACCTC_LUNG_T34      tLung
   TGGACGCTCATGGTCA_LUNG_T34      tLung
   ACTTGTTTCAGCTGTGC_LUNG_T34      tLung
   CCTTCGATCATACGGT_LUNG_T34      tLung
   TGCCTGGGTGTGAAAT_LUNG_T34      tLung
Name: Sample_Origin, Length: 1996, dtype: category
Categories (2, object): ['nLung', 'tLung']
```

```
adata.var
→
```

	var_names
A1BG	A1BG
A1BG-AS1	A1BG-AS1
A1CF	A1CF
A2M	A2M

A2M-AS1	A2M-AS1
...	...
ZYX	ZYX
ZZEF1	ZZEF1
ZZZ3	ZZZ3
bP-21264C1.2	bP-21264C1.2
bP-218909.3	bP-218909.3

29634 rows × 1 columns

Quality Control

adata.obs

	Index		Barcode	
AGTCTTTGTGACTCAT_LUNG_N01	AGTCTTTGTGACTCAT_LUNG_N01	AGTCTTTGTGACTCAT	LUN	
ATCACGACAAGCGCTC_LUNG_N01	ATCACGACAAGCGCTC_LUNG_N01	ATCACGACAAGCGCTC	LUN	
CACAAACAGAAGGCCT_LUNG_N01	CACAAACAGAAGGCCT_LUNG_N01	CACAAACAGAAGGCCT	LUN	
CAGGTGCAGAGACTAT_LUNG_N01	CAGGTGCAGAGACTAT_LUNG_N01	CAGGTGCAGAGACTAT	LUN	
CATTCGCAGGCATGTG_LUNG_N01	CATTCGCAGGCATGTG_LUNG_N01	CATTCGCAGGCATGTG	LUN	
...	...	...	...	
TATCAGGGTCAAAGCTC_LUNG_T34	TATCAGGGTCAAAGCTC_LUNG_T34	TATCAGGGTCAAAGCTC	LUN	
TGGACGCTCATGGTCA_LUNG_T34	TGGACGCTCATGGTCA_LUNG_T34	TGGACGCTCATGGTCA	LUN	
ACTTGTTTCAGCTGTGC_LUNG_T34	ACTTGTTTCAGCTGTGC_LUNG_T34	ACTTGTTTCAGCTGTGC	LUN	

Estimate metrics related to the number of counts per cell, number of genes per cell and mitochondrial fraction per cell

```

# Quality control - calculate QC covariates
adata.obs['n_counts'] = adata.X.sum(1)
adata.obs['log_counts'] = np.log(adata.obs['n_counts'])
adata.obs['n_genes'] = (adata.X > 0).sum(1) # Creates a boolean matrix, so it sums only True

# QC covariates related to mitochondrial genes
mito_genes = [gene for gene in adata.var_names if gene.startswith('MT-')]
adata.obs['mito_counts'] = adata[:, mito_genes].X.sum(axis=1)
adata.obs['mt_frac'] = adata.obs['mito_counts'] / adata.obs['n_counts']

```

```
# Different way to do it, using the function calculate_qc_metrics from scanpy library
sc.pp.calculate_qc_metrics(adata, inplace=True)
```

```
# Check the new columns we have in adata.obs (metadata from cells)
adata.obs
```



	Index	Barcode
AGTCTTTGTGACTCAT_LUNG_N01	AGTCTTTGTGACTCAT_LUNG_N01	AGTCTTTGTGACTCAT LUN
ATCACGACAAGCGCTC_LUNG_N01	ATCACGACAAGCGCTC_LUNG_N01	ATCACGACAAGCGCTC LUN
CACAAACAGAAGGCCT_LUNG_N01	CACAAACAGAAGGCCT_LUNG_N01	CACAAACAGAAGGCCT LUN
CAGGTGCAGAGACTAT_LUNG_N01	CAGGTGCAGAGACTAT_LUNG_N01	CAGGTGCAGAGACTAT LUN
CATTCGCAGGCATGTG_LUNG_N01	CATTCGCAGGCATGTG_LUNG_N01	CATTCGCAGGCATGTG LUN
...	...	...
TATCAGGGTCAAACCTC_LUNG_T34	TATCAGGGTCAAACCTC_LUNG_T34	TATCAGGGTCAAACCTC LUI
TGGACGCTCATGGTCA_LUNG_T34	TGGACGCTCATGGTCA_LUNG_T34	TGGACGCTCATGGTCA LUI
ACTTGTTTCAGCTGTGC_LUNG_T34	ACTTGTTTCAGCTGTGC_LUNG_T34	ACTTGTTTCAGCTGTGC LUI
CCTTCGATCATACGGT_LUNG_T34	CCTTCGATCATACGGT_LUNG_T34	CCTTCGATCATACGGT LUI
TGCGTGGGTGTGAAAT_LUNG_T34	TGCGTGGGTGTGAAAT_LUNG_T34	TGCGTGGGTGTGAAAT LUI

1996 rows × 20 columns

Generate plots to visualize QC metrics and make decisions

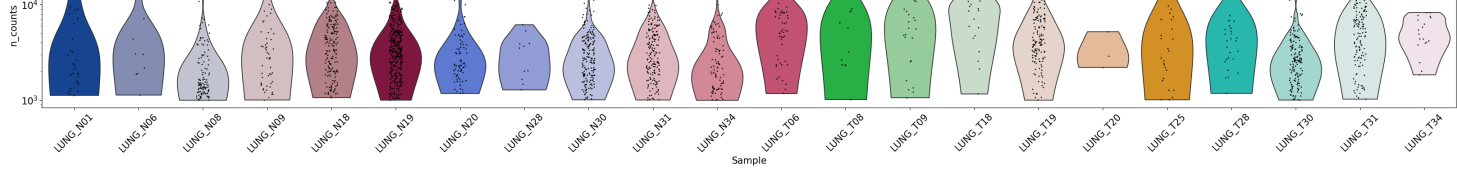
```
import matplotlib.pyplot as plt
```

```
# Quality control - plot QC metrics
# n_counts per Sample quality plots
plt.rcParams['figure.figsize'] = [30,5]
plt.rcParams['font.size'] = '15'
plt.figure()
sc.pl.violin(adata, 'n_counts', groupby='Sample', size=2, log=True, cut=0, show=False)
ax = plt.gca()
ax.set_xticklabels(ax.get_xticklabels(), rotation = 45)
plt.show()
```



```
<ipython-input-37-f7d51541c58f>:8: UserWarning: FixedFormatter should only be used to
ax.set_xticklabels(ax.get_xticklabels(), rotation = 45)
<Figure size 3000x500 with 0 Axes>
```

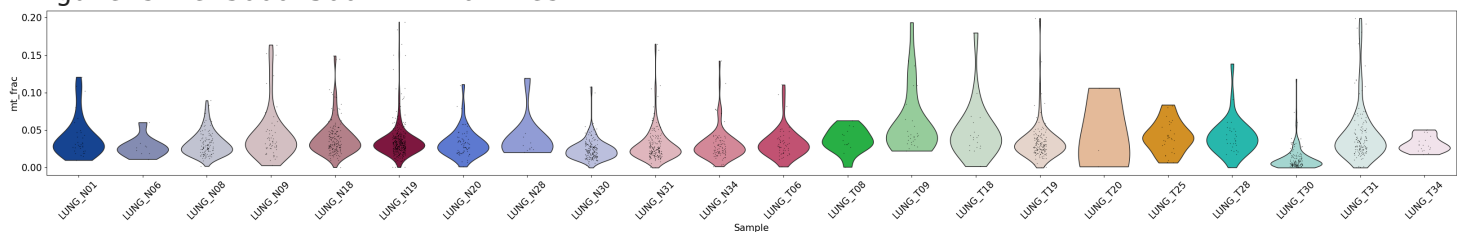




- This plot (above) represents the number of counts per cell categorized by sample. All samples show a good distribution of `n_counts`, being the lower cutoff around 1000. The fact that some samples, as LUNG_T20, have few cells it's because our dataset is tiny (around 2000 cells)

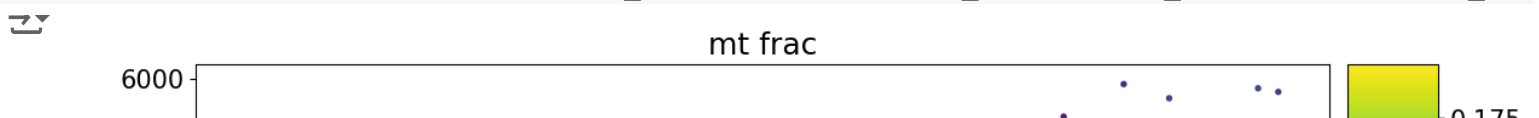
```
# Mitochondrial fraction per sample quality plots
plt.rcParams['figure.figsize'] = [30,5]
plt.figure()
sc.pl.violin(adata, 'mt_frac', groupby='Sample', show = False)
ax = plt.gca()
ax.set_xticklabels(ax.get_xticklabels(), rotation = 45)
plt.show()
```

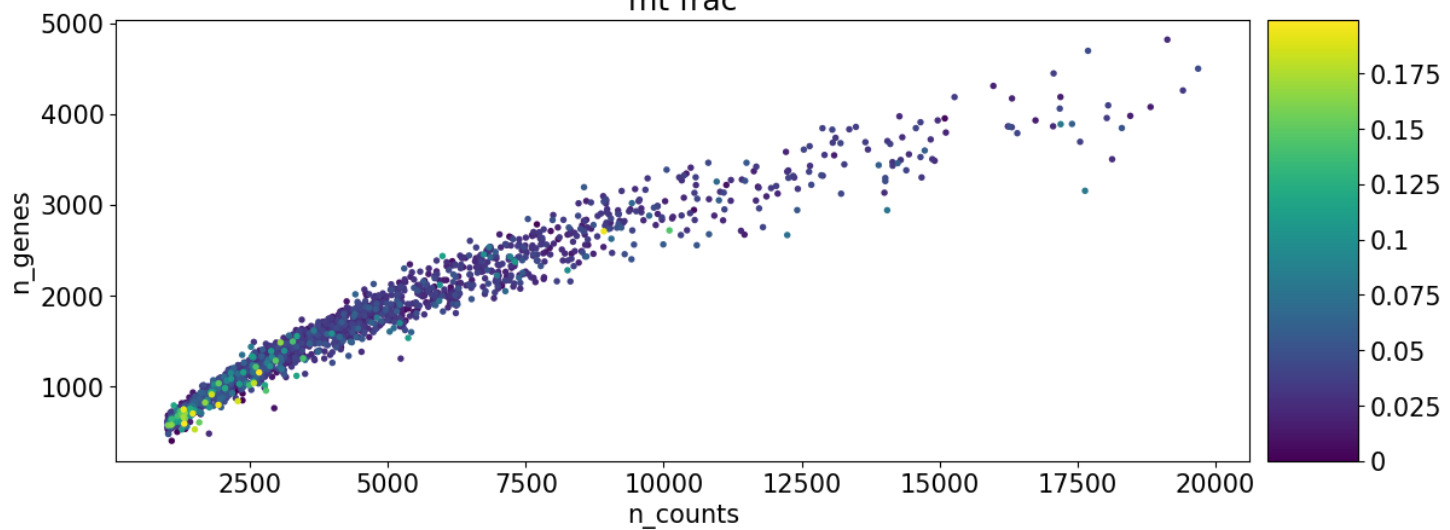
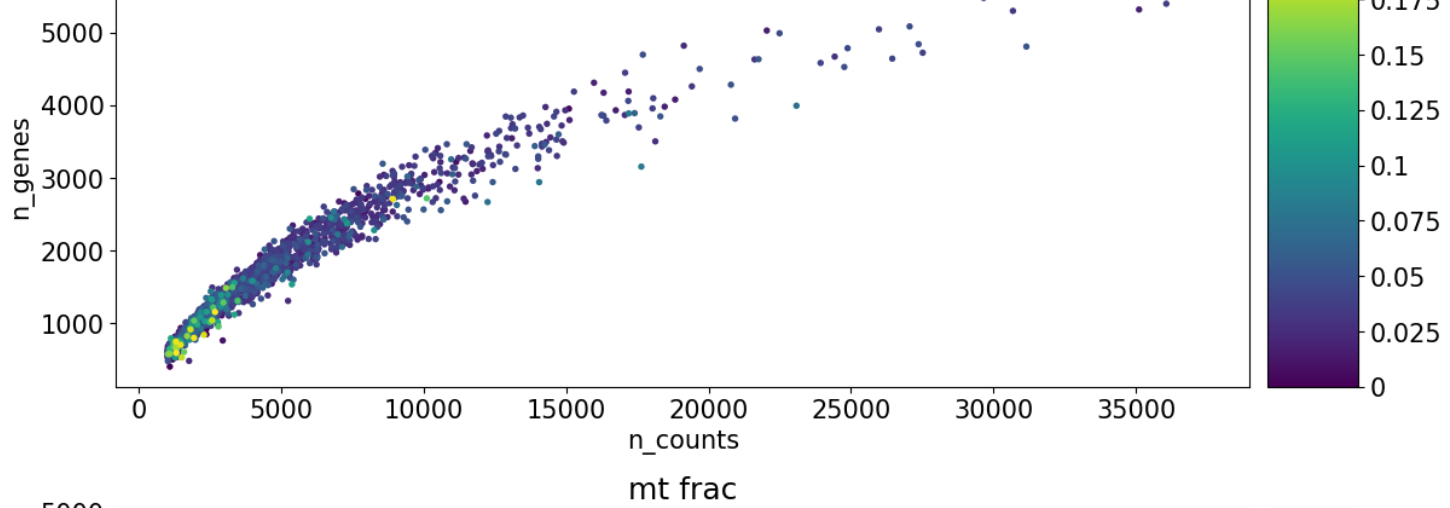
<ipython-input-38-247c967d7ce7>:6: UserWarning: FixedFormatter should only be used to ax.set_xticklabels(ax.get_xticklabels(), rotation = 45)
<Figure size 3000x500 with 0 Axes>



Usually, we use a threshold for mitochondrial fraction lower, at least, than 0.2. As we are plotting raw data, why do you think the top cutoff in the plot is 0.2?

```
#Data quality summary plots
plt.rcParams['figure.figsize'] = [10,5]
p1 = sc.pl.scatter(adata, 'n_counts', 'n_genes', color='mt_frac')
p2 = sc.pl.scatter(adata[adata.obs['n_counts']<20000], 'n_counts', 'n_genes', color='mt_fr
```





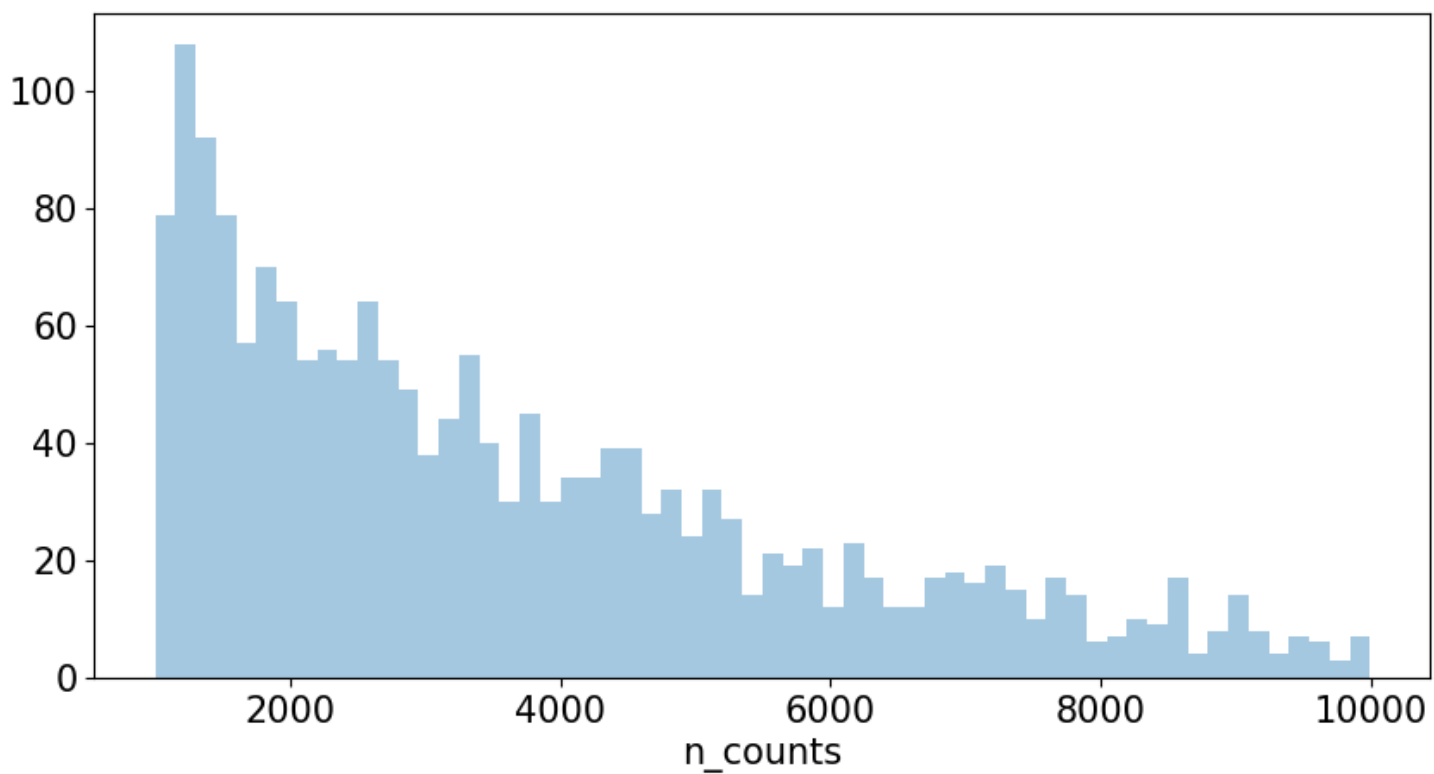
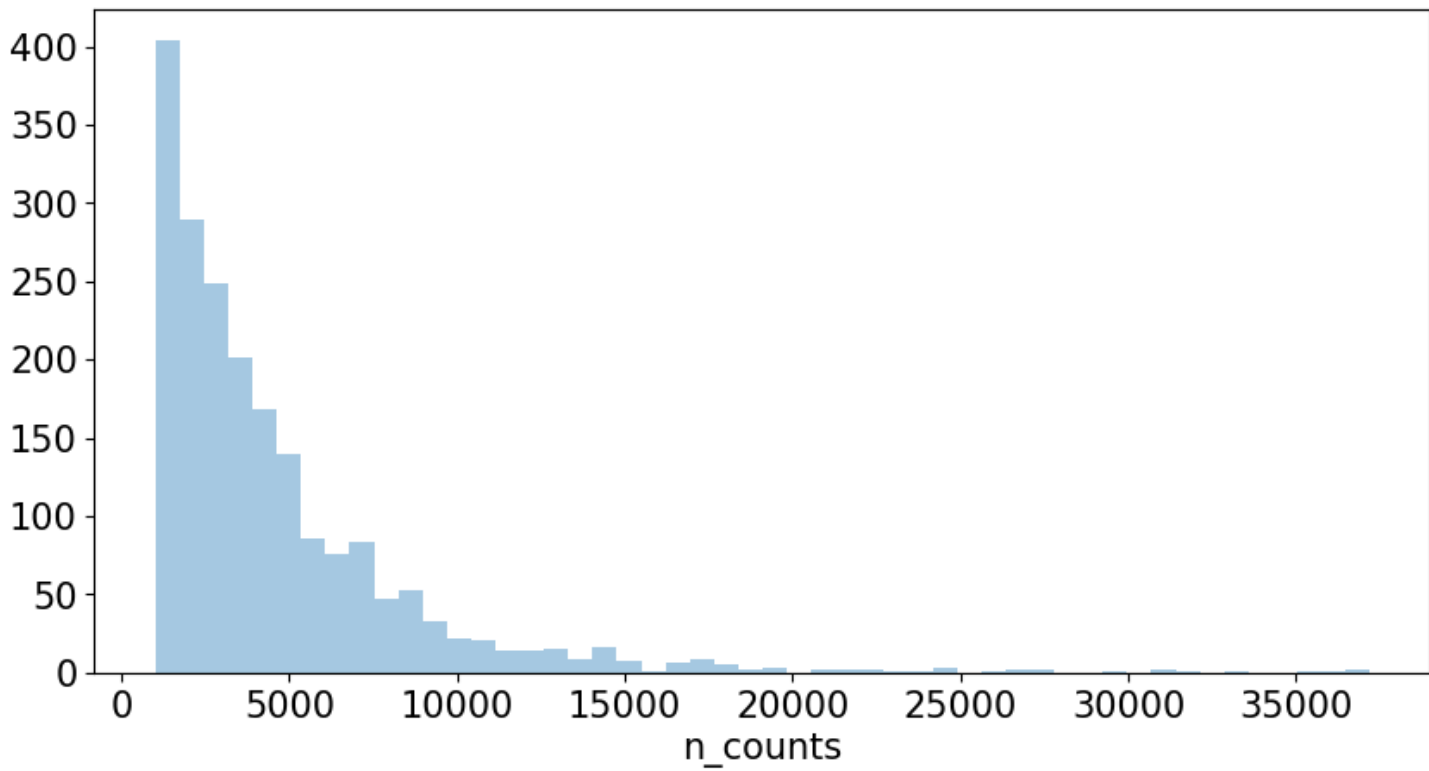
```
import warnings
warnings.filterwarnings('ignore')
```

```
#Thresholding decision: counts
p3 = sb.distplot(adata.obs['n_counts'], kde=False)
plt.show()
```

```
p4 = sb.distplot(adata.obs['n_counts'][adata.obs['n_counts']<10000], kde=False, bins=60)
plt.show()
```

```
#p5 = sb.distplot(adata.obs['n_counts'][adata.obs['n_counts']>40000], kde=False, bins=60)  
#plt.axvline(x = 90000, color = 'r', linestyle = '--')
```

```
plt.show()
```



The y-axis represent the frequency of each value


```
#Thresholding decision: genes
```

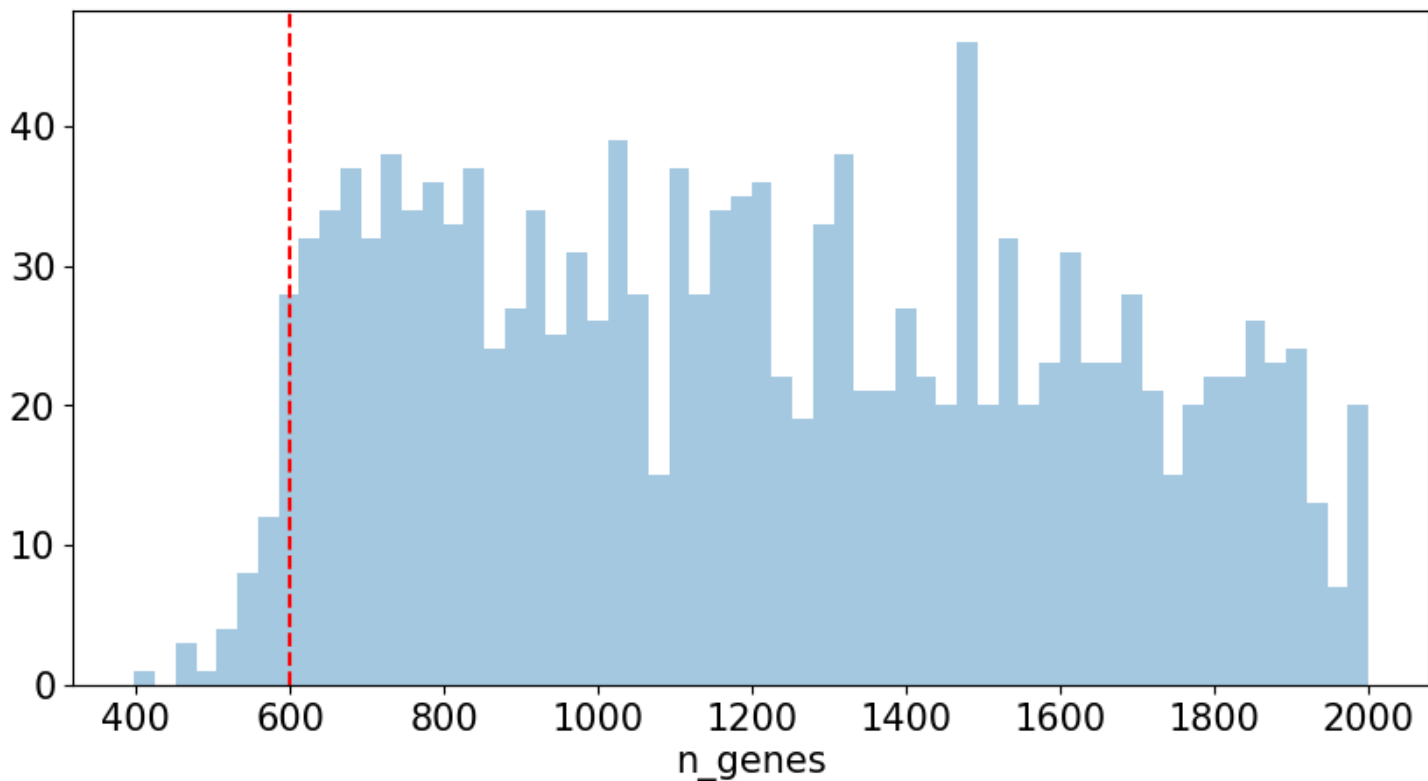
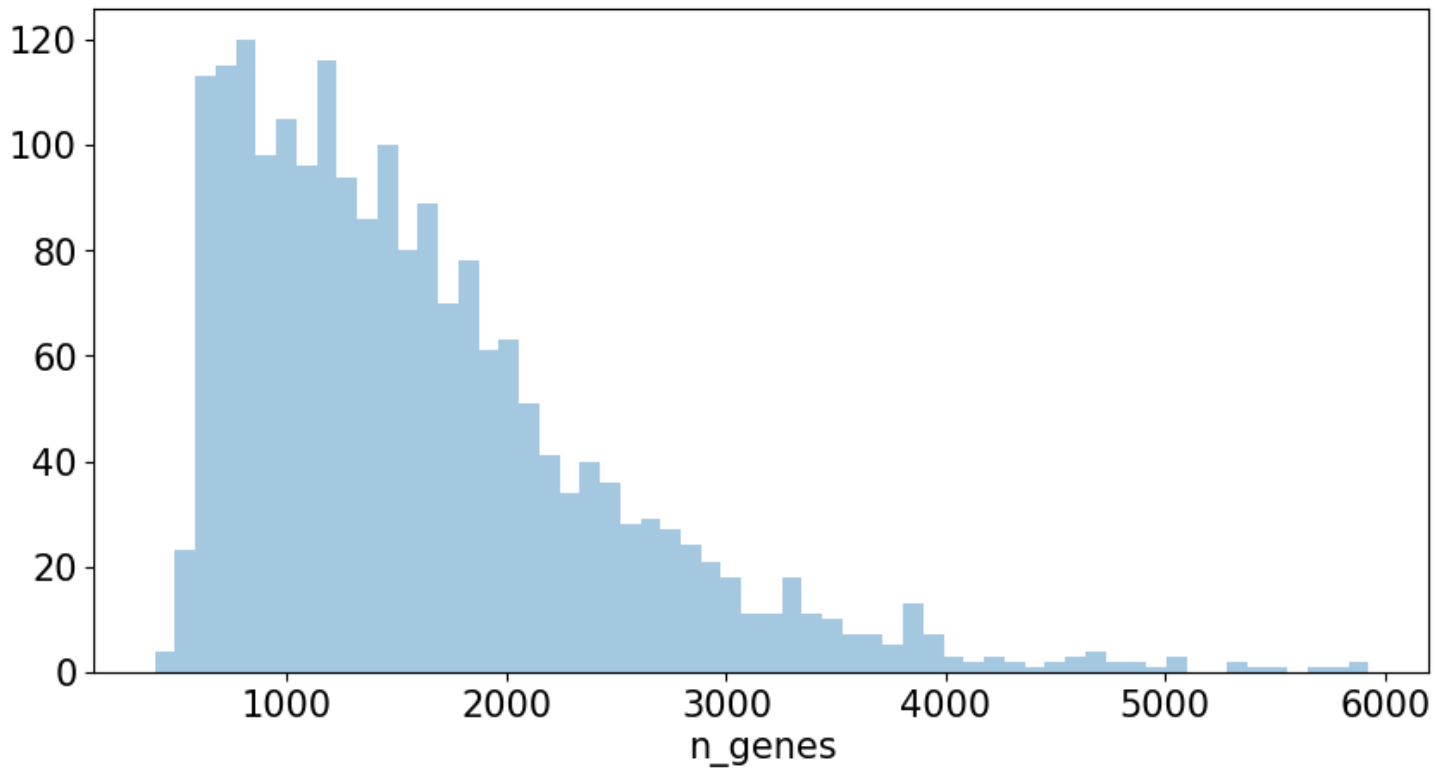
```
p6 = sb.distplot(adata.obs['n_genes'], kde=False, bins=60)
```

```
plt.show()
```

```
p7 = sb.distplot(adata.obs['n_genes'][adata.obs['n_genes']<2000], kde=False, bins=60)
```

```
plt.axvline(x = 600, color = 'r', linestyle = '--')
```

```
plt.show()
```



```
# Filter cells according to identified QC thresholds:
```

```

print('Total number of cells: {:d}'.format(adata.n_obs))

# sc.pp.filter_cells(adata, min_counts = 1000) Already filtered previously!
print('Number of cells after min count filter: {:d}'.format(adata.n_obs))

# sc.pp.filter_cells(adata, max_counts = 90000) # Not necessary because this subset satisfies
print('Number of cells after max count filter: {:d}'.format(adata.n_obs))

adata = adata[adata.obs['mt_frac'] < 0.2] # We can decrease to 0.15
print('Number of cells after MT filter: {:d}'.format(adata.n_obs))

sc.pp.filter_cells(adata, min_genes = 600)
print('Number of cells after gene filter: {:d}'.format(adata.n_obs))

```

Total number of cells: 1996
 Number of cells after min count filter: 1996
 Number of cells after max count filter: 1996
 Number of cells after MT filter: 1996
 Number of cells after gene filter: 1953

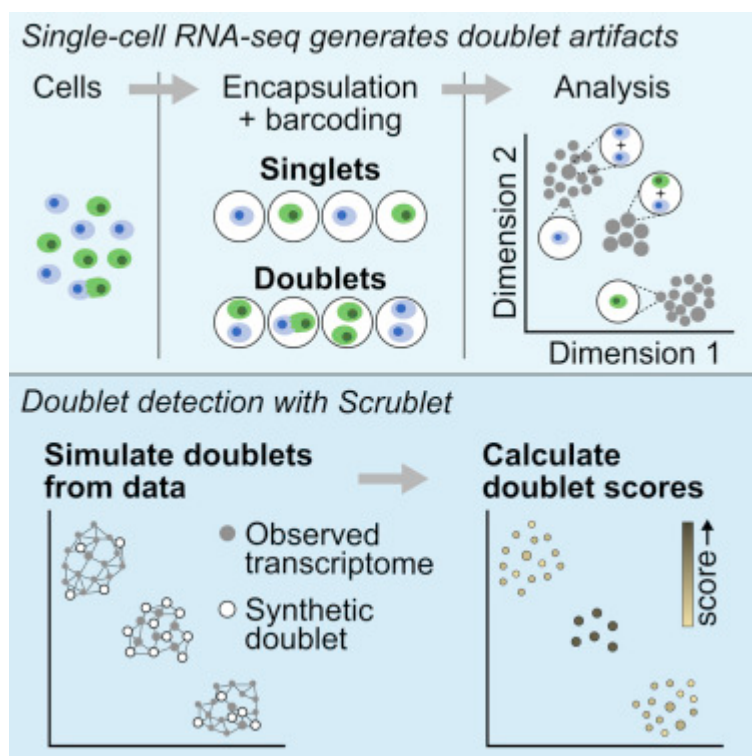
```

# Filtering genes
prefilter_var = int(adata.n_vars)
print(f'Original number of genes:', prefilter_var)
sc.pp.filter_genes(adata, min_cells=10)
print(f'Number of genes after quality control:{adata.n_vars}')
print(f'Number of genes removed:', prefilter_var-adata.n_vars)

```

Original number of genes: 29634
 Number of genes after quality control:13609
 Number of genes removed: 16025

Doublets detection



```

# Installing the package
!pip install scrublet

Collecting scrublet
  Downloading scrublet-0.2.2-py2.py3-none-any.whl (15.4 kB)

```

```

Downloading scrublet-0.2.3-py3-none-any.whl (15 kB)
Requirement already satisfied: cython in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: scikit-image in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (
Collecting annoy (from scrublet)
  Downloading annoy-1.17.3.tar.gz (647 kB)
    647.5/647.5 kB 5.6 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from
Requirement already satisfied: umap-learn in /usr/local/lib/python3.10/dist-packages (
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: llvmlite<0.42,>=0.41.0dev0 in /usr/local/lib/python3.10
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: networkx>=2.2 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: imageio>=2.4.1 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: tifffile>=2019.7.26 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: PyWavelets>=1.1.1 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: pynndescent>=0.5 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from u
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (fr
Building wheels for collected packages: annoy
  Building wheel for annoy (setup.py) ... done
  Created wheel for annoy: filename=annoy-1.17.3-cp310-cp310-linux_x86_64.whl size=552
  Stored in directory: /root/.cache/pip/wheels/64/8a/da/f714bcf46c5efdcfcac0559e63370c
Successfully built annoy
Installing collected packages: annoy, scrublet
Successfully installed annoy-1.17.3 scrublet-0.2.3

```

```
import scrublet as scr
```

```

# Include default parameters
scrub = scr.Scrublet(adata.X.tocsc(), expected_doublet_rate=0.06)
doublet_scores, predicted_doublets = scrub.scrub_doublets(min_counts=2,
                                                         min_cells=3,
                                                         min_gene_variability_pctl=85,
                                                         n_prin_comps=30)

# Add the columns with doublet_scores and predicted_doublets results to adata.obs
adata.obs['Doublet_score']=doublet_scores
adata.obs['Predicted_doublet']=predicted_doublets
adata.obs['Doublet_Threshold']=scrub.threshold_

```

```

Preprocessing...
Simulating doublets...
Embedding transcriptomes using PCA...
Calculating doublet scores...
Automatically set threshold at doublet score = 0.46
Detected doublet rate = 0.0%
Estimated detectable doublet fraction = 8.1%

```

```
Overall doublet rate:
    Expected    = 6.0%
    Estimated   = 0.0%
Elapsed time: 4.8 seconds
```

```
print(adata.obs['Predicted_doublet'].value_counts())
print('\nWe do not find any doublets!')
```

```
Predicted_doublet
False          1953
Name: count, dtype: int64
```

```
We do not find any doublets!
```

✓ Normalization

```
# Save in layers the raw matrix
adata.layers['counts'] = adata.X
```

```
print(adata.X.max())

3777.0
```

```
# Perform normalization. Normalization is necessary to account for differences in sequencing depth.
# these technical variations can obscure biological signals.
import gc
gc.collect()
sc.pp.normalize_per_cell(adata, counts_per_cell_after=1e6) # This step adjusts for differences in sequencing depth
sc.pp.log1p(adata) # Apply a logarithmic transformation to the normalized counts to stabilize variance
gc.collect()

13348
```

```
print(adata.X.max())

12.832525
```

✓ Downstream analysis

✓ Read the files barcode.csv, varnames.csv and matrix.gz (normalized one)

```
del adata
```

```
# Normalized data, barcodes metadata, genes metadata
if not os.path.exists('endothelial_matrix.npz'):
    !wget --no-check-certificate 'https://docs.google.com/uc?export=download&id=1AWnvjuruS'
if not os.path.exists('endothelial_barcodes.csv'):
    !wget https://raw.githubusercontent.com/nunososorio/SingleCellGenomics2024/main/challenge_data/endothelial_barcodes.csv
if not os.path.exists('endothelial_varnames.csv'):
    !wget https://raw.githubusercontent.com/nunososorio/SingleCellGenomics2024/main/challenge_data/endothelial_varnames.csv
```

```
matrix = sparse.load_npz('endothelial_matrix.npz')
```

```
barcodes = pd.read_csv('endothelial_barcodes.csv')
varnames = pd.read_csv('endothelial_varnames.csv')
```

▼ Create the anndata object

```
adata = sc.AnnData(X=matrix, obs=barcodes, var=varnames)
```

```
adata.obs_names = list(adata.obs['obs_names'])
adata.var_names = list(adata.var['var_names'])
```

```
adata
```

```
AnnData object with n_obs × n_vars = 2107 × 20599
  obs: 'Unnamed: 0', 'Barcode', 'barcode', 'cellsubtype', 'celltype',
'celltype_refined', 'index', 'n_counts', 'obs_names', 'origin', 'sample'
  var: 'Unnamed: 0', 'dispersions', 'dispersions_norm', 'highly_variable', 'means',
'n_cells', 'var_names'
```

```
adata.obs['origin'].astype('category').value_counts()
```

```
origin
nLung      1295
tLung       627
mBrain      159
tL/B        18
mLN          8
Name: count, dtype: int64
```

▼ How many cells and genes do we have?

```
adata.n_obs
```

```
2107
```

```
adata.n_vars
```

```
20599
```

▼ How many samples do we have?

```
adata.obs['sample'] = adata.obs['sample'].astype('category')
```

```
len(adata.obs['sample'].cat.categories)
```

```
36
```

▼ Which is the sample with more number of cells? And with lower?

```
adata.obs['sample'].value_counts()
```

```
sample
LUNG_N19      395
LUNG_N30      180
LUNG_N18      173
```

LUNG_N31	164
LUNG_T19	159
LUNG_T30	145
LUNG_T31	135
LUNG_N08	112
LUNG_N34	88
LUNG_N20	75
NS_04	58
LUNG_N09	57
LUNG_T06	51
LUNG_N01	32
LUNG_T25	30
LUNG_T28	29
LUNG_T09	28
NS_19	27
NS_07	24
NS_17	20
LUNG_T34	19
LUNG_T18	17
BRONCHO_58	12
LUNG_T08	12
NS_12	12
LUNG_N28	11
LUNG_N06	8
NS_03	7
NS_13	5
EBUS_10	4
EBUS_28	4
BRONCHO_11	4
NS_06	3
NS_16	3
EBUS_06	2
LUNG_T20	2

Name: count, dtype: int64

Determine HVGs. In this case, select as top genes the 10% of the total genes

HVGs are more likely to capture the biological heterogeneity among cells. These genes show significant variability across different cells, reflecting underlying biological processes such as cell type differences, cell states, or responses to stimuli. By focusing on HVGs, we emphasize biologically meaningful variations and reduce the impact of technical noise and less informative genes.

```
sc.pp.highly_variable_genes(adata, flavor='seurat', n_top_genes=int(adata.n_vars*0.1))
print('\n', 'Number of highly variable genes: {:d}'.format(np.sum(adata.var['highly_variable'])))
```

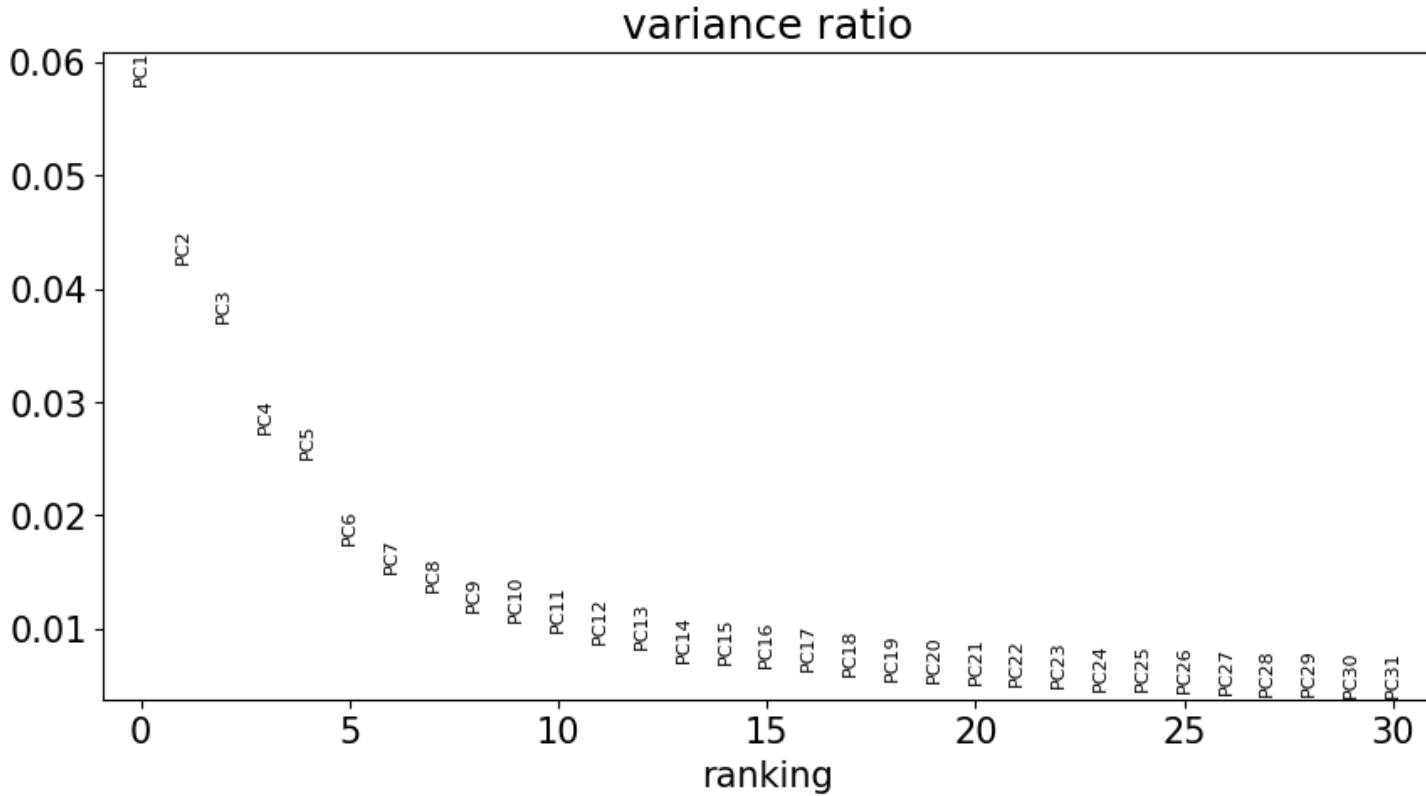
```
Number of highly variable genes: 2059
```

Dimensionality reduction

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms high-

dimensional data into a lower-dimensional form, capturing the most significant variance in the data. PCA identifies the directions (principal components) along which the variation in the data is maximized. Each PC is a linear combination of the original gene expression levels. The first PC captures the direction of maximum variance in the data. The second PC captures the next highest variance orthogonal to the first, and so on.

```
sc.pp.pca(adata, n_comps=50, use_highly_variable=True, svd_solver='arpack')
sc.pl.pca_variance_ratio(adata, log=False)
```



```
sc.pp.pca(adata, n_comps=15, use_highly_variable=True, svd_solver='arpack')
```

✖ Neighbors

Identifying groups of similar cells (clusters) based on their gene expression profiles.

Estimate neighbors with size of the local neighborhood = 10

```
sc.pp.neighbors(adata, n_neighbors=10, n_pcs=15)
```

✖ Dimensionality reduction (visualization)

It uses the neighbor graph to project high-dimensional data into lower dimensions while preserving the

It uses the neighbor graph to project high dimensional data into lower dimensions while preserving the local structure.

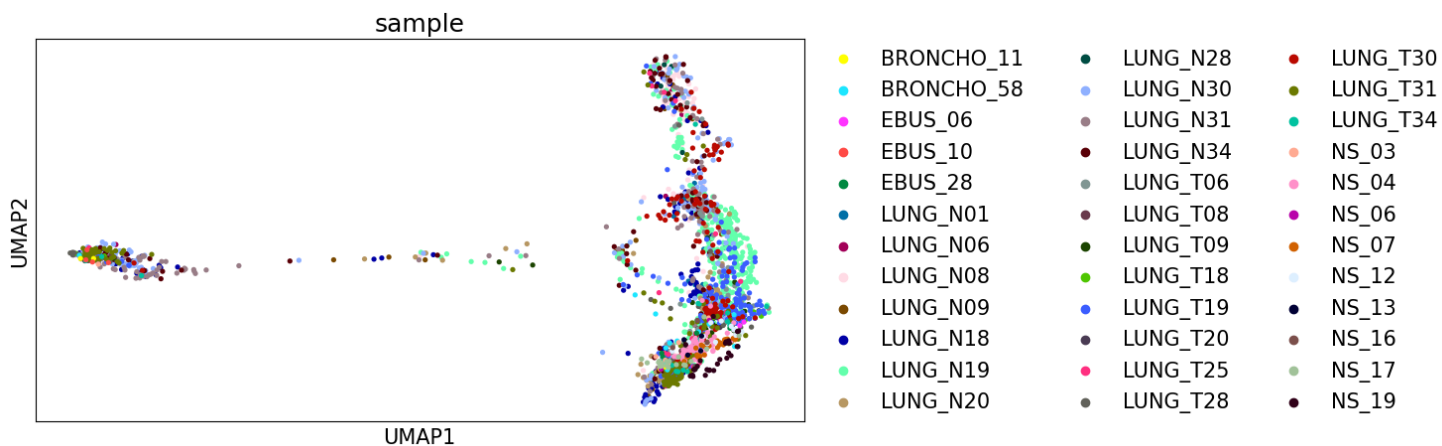
```
sc.tl.umap(adata, n_components=15, alpha=0.3, min_dist=0.5, random_state=42)
sc.tl.tsne(adata, n_pcs = 15)
```

adata

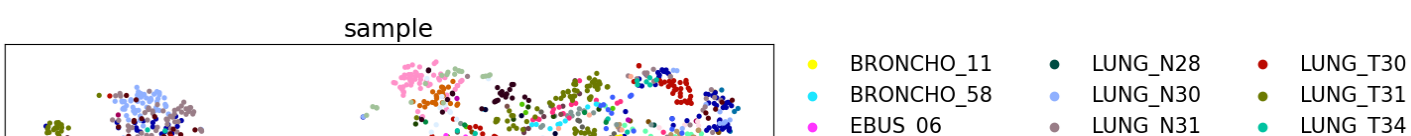
```
AnnData object with n_obs × n_vars = 2107 × 20599
  obs: 'Unnamed: 0', 'Barcode', 'barcode', 'cellsubtype', 'celltype',
'celltype_refined', 'index', 'n_counts', 'obs_names', 'origin', 'sample'
  var: 'Unnamed: 0', 'dispersions', 'dispersions_norm', 'highly_variable', 'means',
'n_cells', 'var_names'
  uns: 'hvg', 'pca', 'neighbors', 'umap', 'tsne'
  obsm: 'X_pca', 'X_umap', 'X_tsne'
  varm: 'PCs'
  obsp: 'distances', 'connectivities'
```

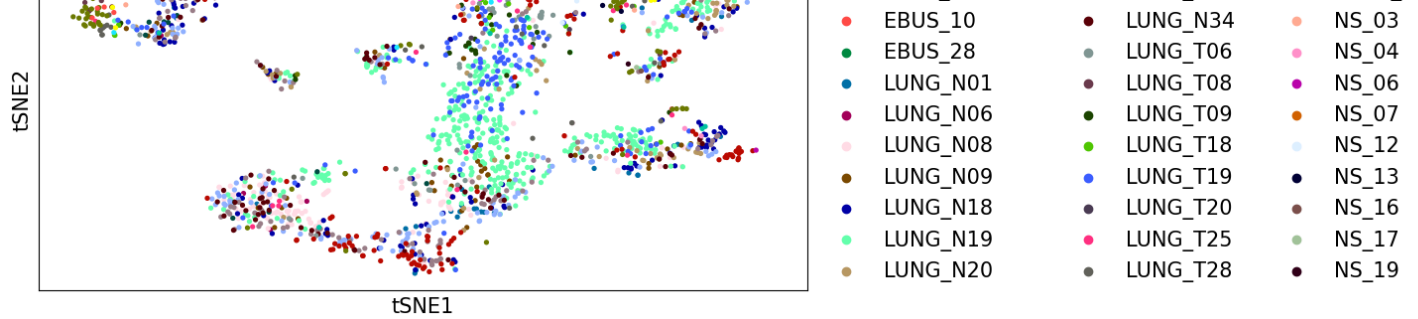
✓ Is there any bias related to sample origin? E.g: It seems like a group of cells from mLN samples cluster together

```
plt.rcParams['figure.facecolor'] = 'white'
sc.pl.umap(adata, color = 'sample')
```



```
sc.pl.tsne(adata, color = 'sample')
```





▼ Clustering

```
import sys
IN_COLAB = 'google.colab' in sys.modules

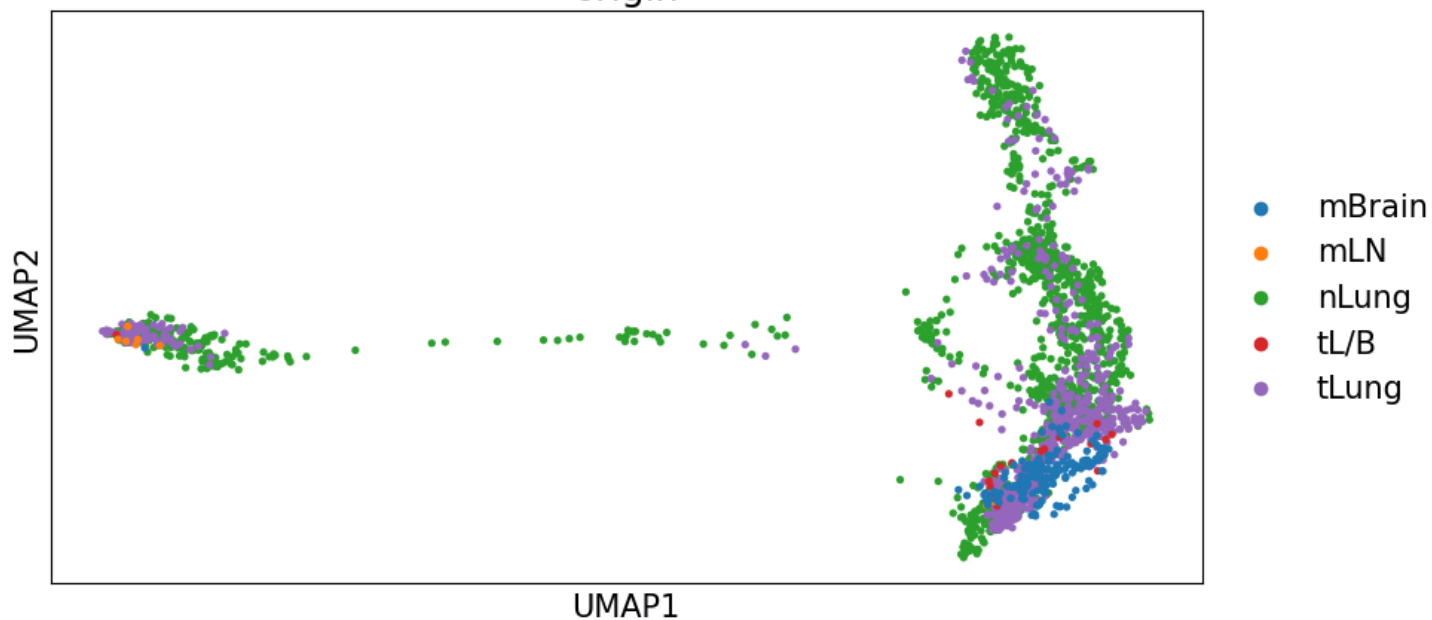
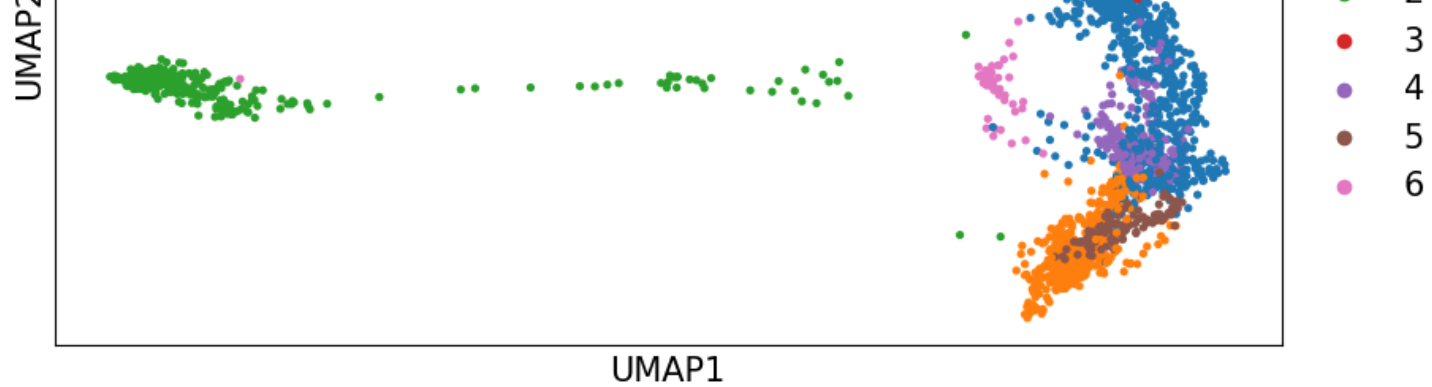
if IN_COLAB:
    !pip install igraph louvain

Collecting igraph
  Downloading igraph-0.11.5-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.3/3.3 MB)
    25.3 MB/s eta 0:00:00
Collecting louvain
  Downloading louvain-0.8.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.1/1.1 MB)
    53.0 MB/s eta 0:00:00
Collecting texttable>=1.6.2 (from igraph)
  Downloading texttable-1.7.0-py2.py3-none-any.whl (10 kB)
Installing collected packages: texttable, igraph, louvain
Successfully installed igraph-0.11.5 louvain-0.8.2 texttable-1.7.0
```

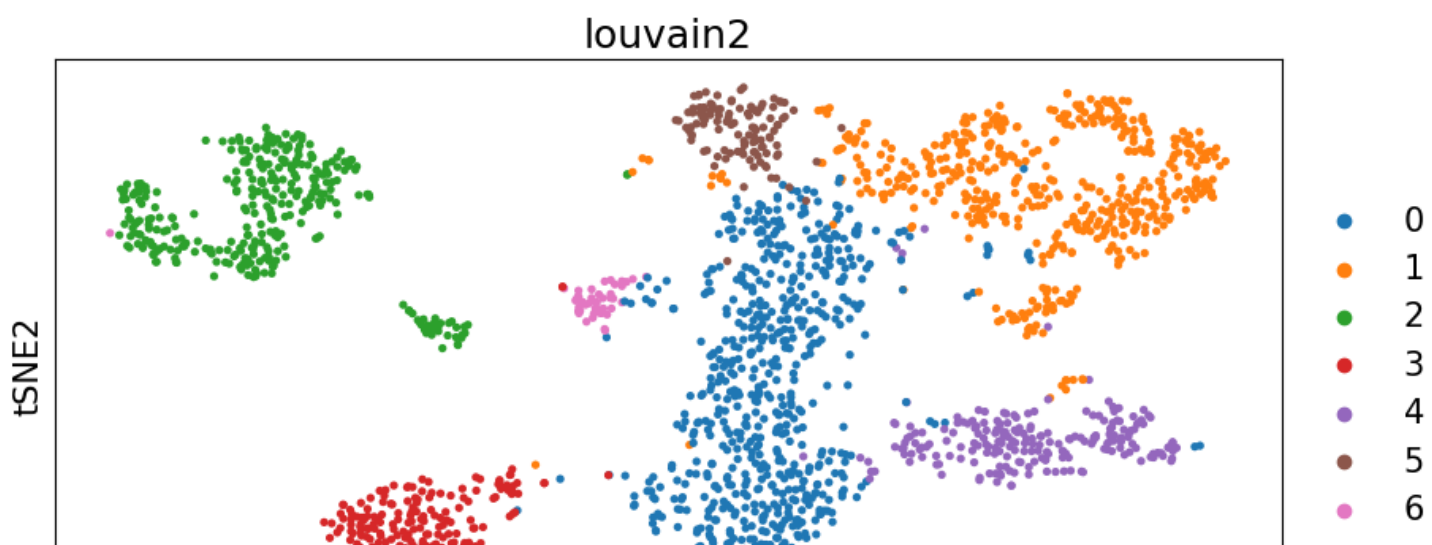
```
sc.tl.louvain(adata, resolution = 0.2, key_added = 'louvain2', random_state = 0) # If we i
# We have to take into account that the resulting populations make sense at biological lev
```

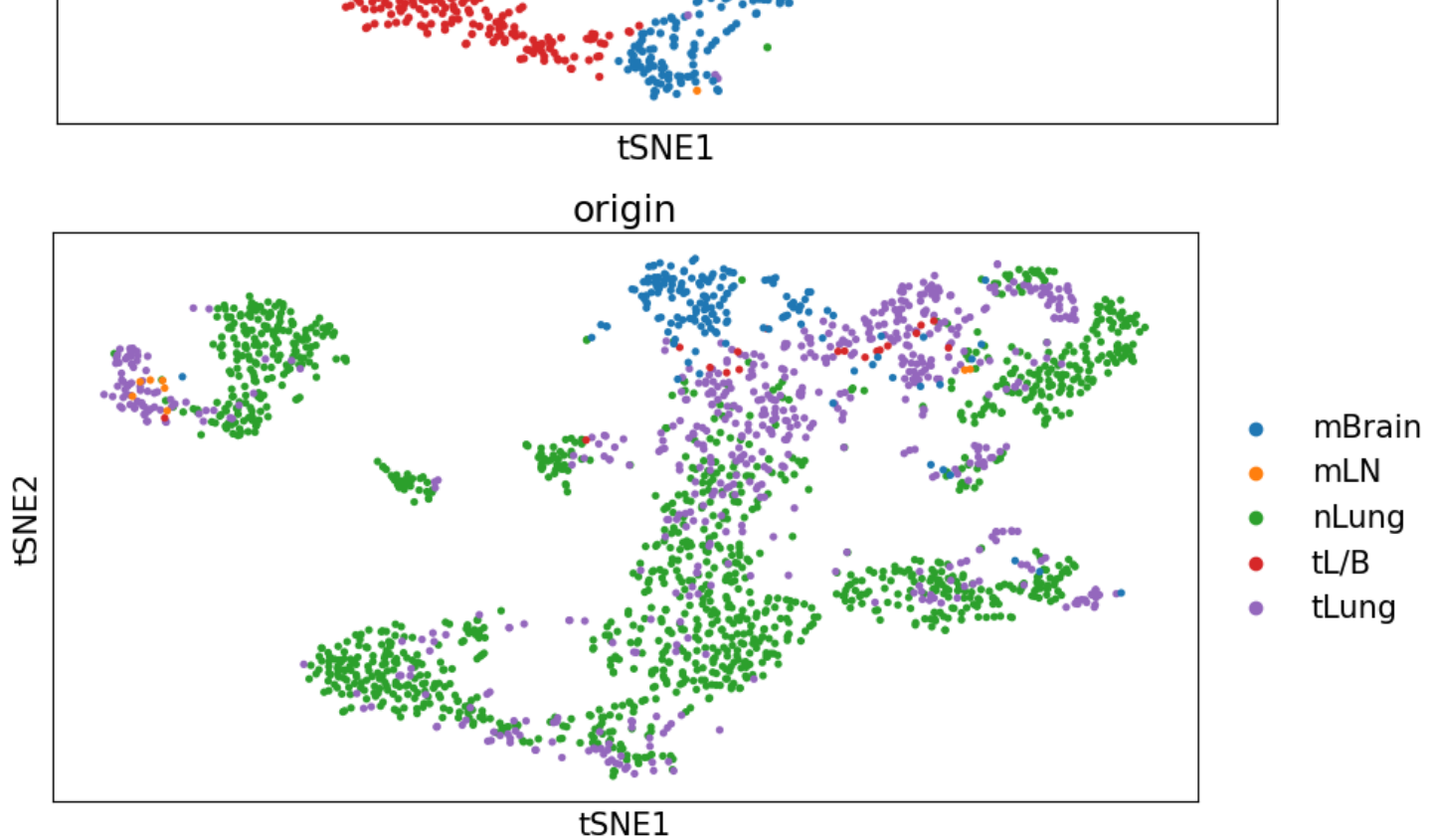
```
# Plot the clusters we find out:
sc.pl.umap(adata, color = 'louvain2')
sc.pl.umap(adata, color = 'origin')
```





```
sc.pl.tsne(adata, color = 'louvain2')
sc.pl.tsne(adata, color = 'origin')
```





adata

```
AnnData object with n_obs × n_vars = 2107 × 20599
  obs: 'Unnamed: 0', 'Barcode', 'barcode', 'cellsubtype', 'celltype',
'celltype_refined', 'index', 'n_counts', 'obs_names', 'origin', 'sample', 'louvain2'
  var: 'Unnamed: 0', 'dispersions', 'dispersions_norm', 'highly_variable', 'means',
'n_cells', 'var_names'
  uns: 'hvg', 'pca', 'neighbors', 'umap', 'tsne', 'sample_colors', 'louvain2',
'louvain2_colors', 'origin_colors'
  obsm: 'X_pca', 'X_umap', 'X_tsne'
  varm: 'PCs'
  obsp: 'distances', 'connectivities'
```

▼ Which is the cluster with more number of cells? How many does it have?

```
adata.obs['louvain2'].value_counts()
```

```
louvain2
0      677
1      482
2      320
3      262
4      212
```

```
5      111
6      43
Name: count, dtype: int64
```

Annotation

Determine the marker genes for each cluster

```
sc.tl.rank_genes_groups(adata, groupby='louvain2', key_added='rank_genes_groups', method='
                        groups = 'all', reference = 'rest')
```

```
gc.collect()

10500
```

Estimate the logfoldchanges for each marker in the resulting clusters and plot the top 10 markers for each one taking into account the logfoldchanges values

```
cero_markers = sc.get.rank_genes_groups_df(adata, group='0', key='rank_genes_groups', pval_
cero_markers['cluster'] = ['0']*len(cero_markers)

uno_markers = sc.get.rank_genes_groups_df(adata, group='1', key='rank_genes_groups', pval_
uno_markers['cluster'] = ['1']*len(uno_markers)

dos_markers = sc.get.rank_genes_groups_df(adata, group='2', key='rank_genes_groups', pval_
dos_markers['cluster'] = ['2']*len(dos_markers)

tres_markers = sc.get.rank_genes_groups_df(adata, group='3', key='rank_genes_groups', pval_
tres_markers['cluster'] = ['3']*len(tres_markers)

cuatro_markers = sc.get.rank_genes_groups_df(adata, group='4', key='rank_genes_groups', pv
cuatro_markers['cluster'] = ['4']*len(cuatro_markers)

cinco_markers = sc.get.rank_genes_groups_df(adata, group='5', key='rank_genes_groups', pva
cinco_markers['cluster'] = ['5']*len(cinco_markers)

seis_markers = sc.get.rank_genes_groups_df(adata, group='6', key='rank_genes_groups', pval_
seis_markers['cluster'] = ['6']*len(seis_markers)
```

Select 'de novo' markers

Cluster number 0

```
cero_markers.sort_values('logfoldchanges', ascending=False)
```

	names	scores	logfoldchanges	pvals	pvals_adj	cluster
0	FCN3	21.828159	4.971938	1.253671e-105	2.582437e-101	0
1	RGCC	21.785065	3.590615	3.214929e-105	3.311216e-101	0

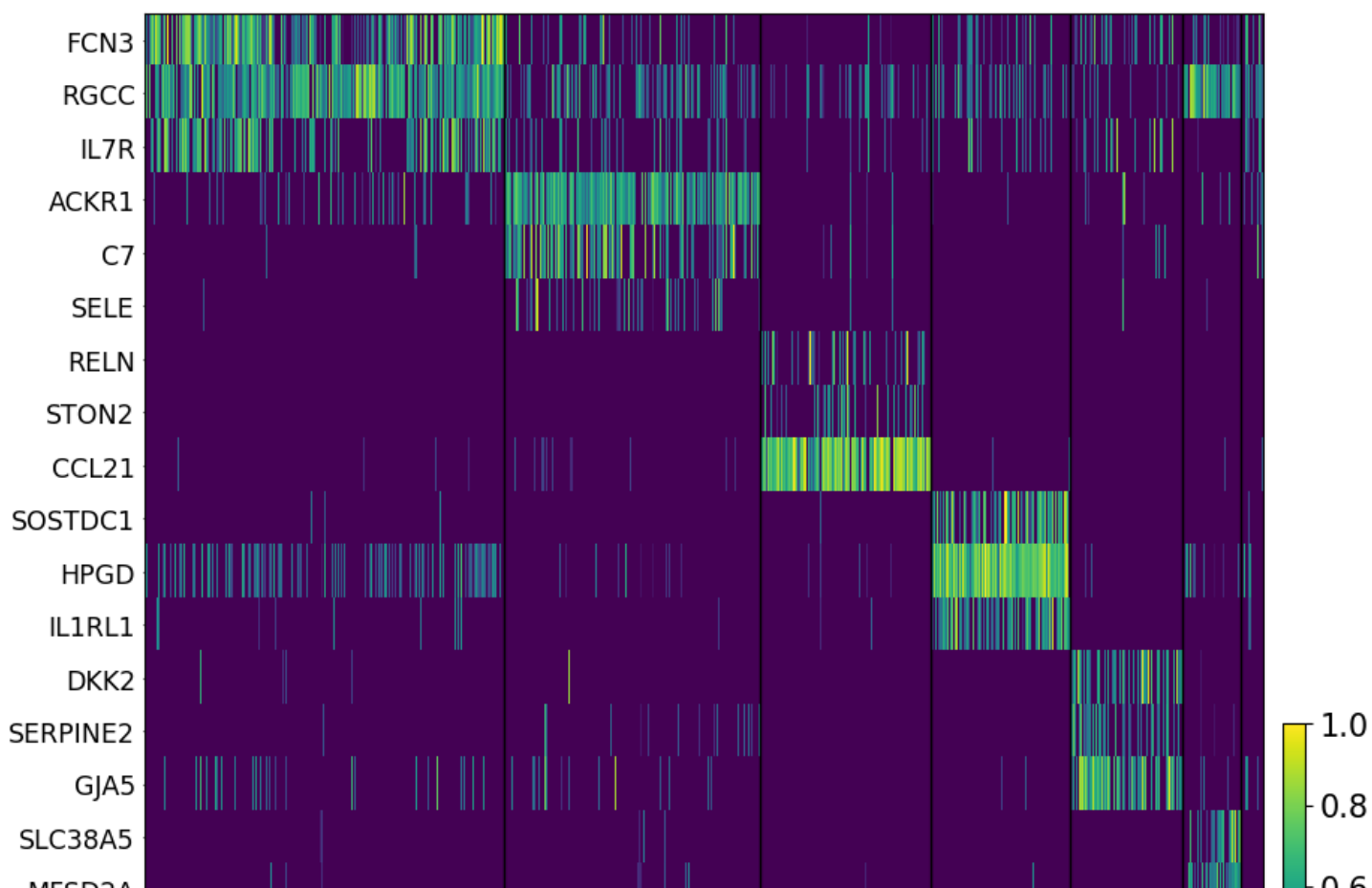
13	IL7R	14.843740	3.588220	7.637569e-50	8.740349e-47	0
11	GPIHBP1	15.309430	3.129344	6.614114e-53	8.515258e-50	0
50	SLC6A4	9.468568	2.990324	2.837057e-21	7.214880e-19	0
...	...	...	...	...	...	...
26	H3F3A	11.869555	0.692082	1.703699e-32	9.485001e-30	0
72	ACTG1	8.466872	0.654348	2.520702e-17	4.476201e-15	0
46	HMGB1	9.770964	0.631969	1.500187e-22	4.162633e-20	0
67	HLA-E	8.882872	0.621063	6.515609e-19	1.315834e-16	0
88	IGFBP4	7.307825	0.606117	2.715008e-13	3.214164e-11	0

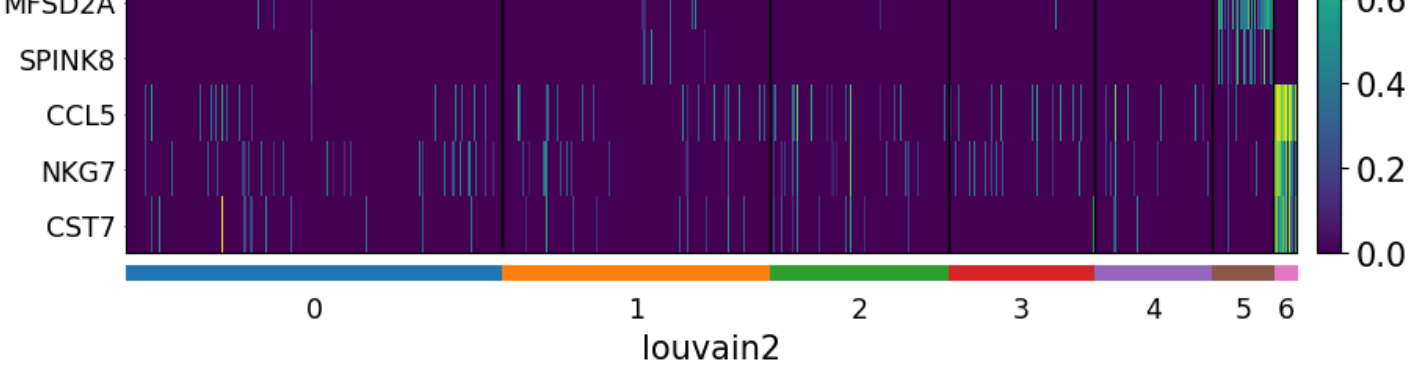
92 rows × 6 columns

✓ Which are the top 3 markers for this cluster? (Taking into account the logfoldchange)

Heatmap with 3 best markers for each cluster

```
sc.pl.heatmap(adata, list(cero_markers.sort_values('logfoldchanges', ascending=False).name
list(uno_markers.sort_values('logfoldchanges', ascending=False).names[0:3])+
list(dos_markers.sort_values('logfoldchanges', ascending=False).names[0:3])+
list(tres_markers.sort_values('logfoldchanges', ascending=False).names[0:3])
list(cuatro_markers.sort_values('logfoldchanges', ascending=False).names[0:3]
list(cinco_markers.sort_values('logfoldchanges', ascending=False).names[0:3]
list(seis_markers.sort_values('logfoldchanges', ascending=False).names[0:3])
, swap_axes=True,
groupby = 'louvain2', standard_scale='var', figsize=[10,10])
```





Markers highlighted in the original article

▼ Markers:

Tumor cells: 'INSR','HSPG2','VWA1'
 Tip-like cells: 'RGCC','RAMP3','ADM'
 Stalk-like cells: 'ACKR1','SELP'
 EPCs: 'TYROBP','C1QB'
 Lymphatic: 'LYVE1','PROX1'

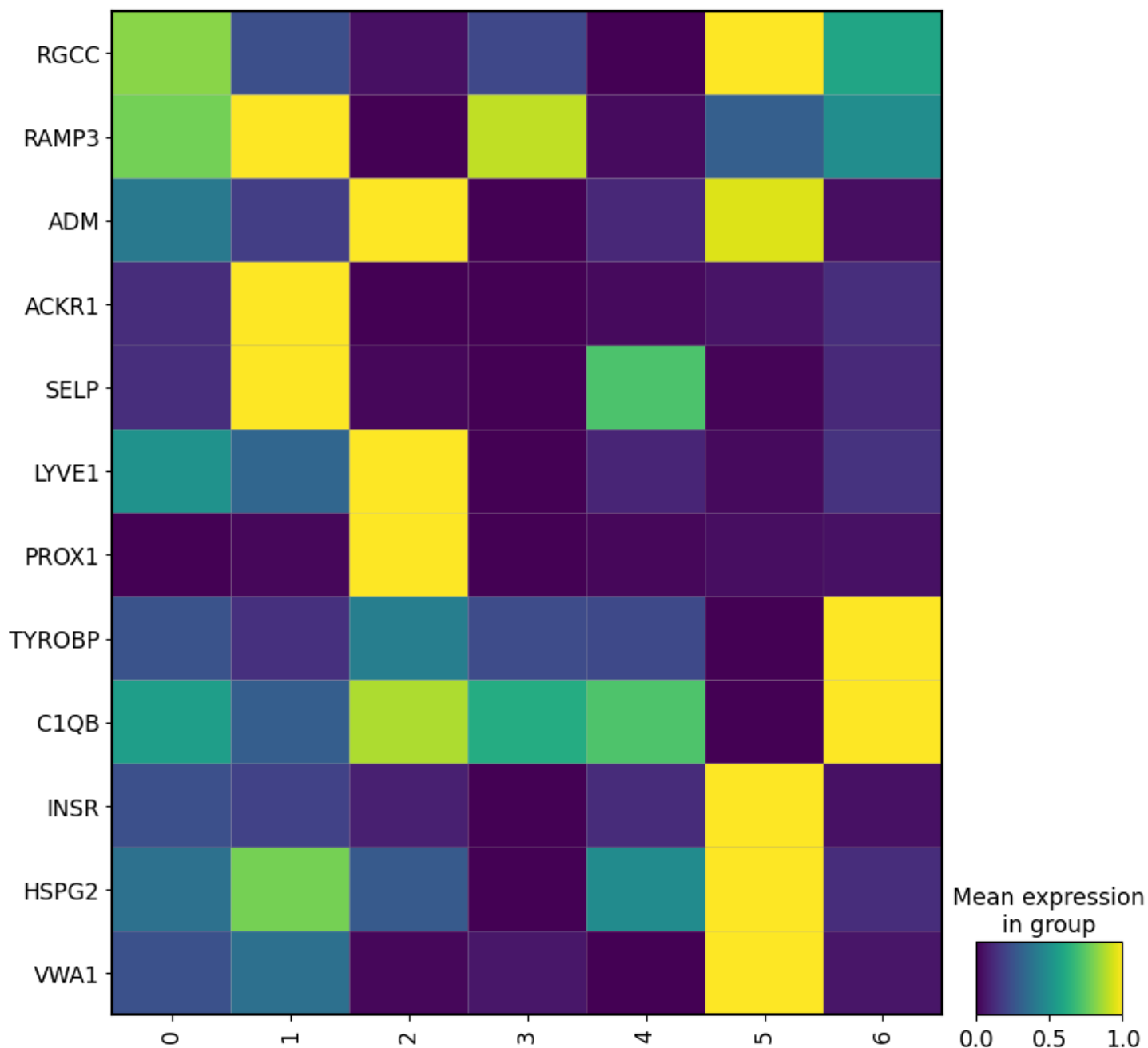
```
markers = pd.concat([cero_markers, uno_markers, dos_markers, tres_markers, cuatro_markers,
```

```
markers[markers.names.str.contains('|'.join(['INSR','HSPG2','VWA1',
                                             'RGCC','RAMP3','ADM',
                                             'ACKR1','SELP',
                                             'TYROBP','C1QB',
                                             'LYVE1','PROX1'])))]
```

	names	scores	logfoldchanges	pvals	pvals_adj	cluster
1	RGCC	21.785065	3.590615	3.214929e-105	3.311216e-101	0
0	ACKR1	27.157953	7.119814	2.039593e-162	4.201357e-158	1
16	HSPG2	14.445927	2.188098	2.659556e-47	2.608771e-44	1
20	SELP	13.940467	3.264275	3.595794e-44	2.554130e-41	1
75	RAMP3	10.298071	1.538975	7.188850e-25	1.526630e-22	1
9	PROX1	18.519579	7.802619	1.435536e-76	1.556347e-73	2
120	LYVE1	10.385641	2.469165	2.882255e-25	3.280198e-23	2
2	INSR	12.376825	4.633728	3.488281e-35	7.083000e-32	5

2	INSR	12.576835	4.655728	5.488281e-33	7.983900e-32	5
11	RGCC	10.545252	3.433777	5.342967e-26	5.240942e-23	5
14	VWA1	10.404675	3.555519	2.360604e-25	2.026087e-22	5
25	HSPG2	9.204885	2.616416	3.420386e-20	1.718452e-17	5

```
sc.pl.matrixplot(adata, ['RGCC','RAMP3','ADM',
                        'ACKR1','SELP',
                        'LYVE1','PROX1',
                        'TYROBP','C1QB',
                        'INSR','HSPG2','VWA1'],
                groupby='louvain2', swap_axes=True, figsize=[10,10], standard_scale='var')
```



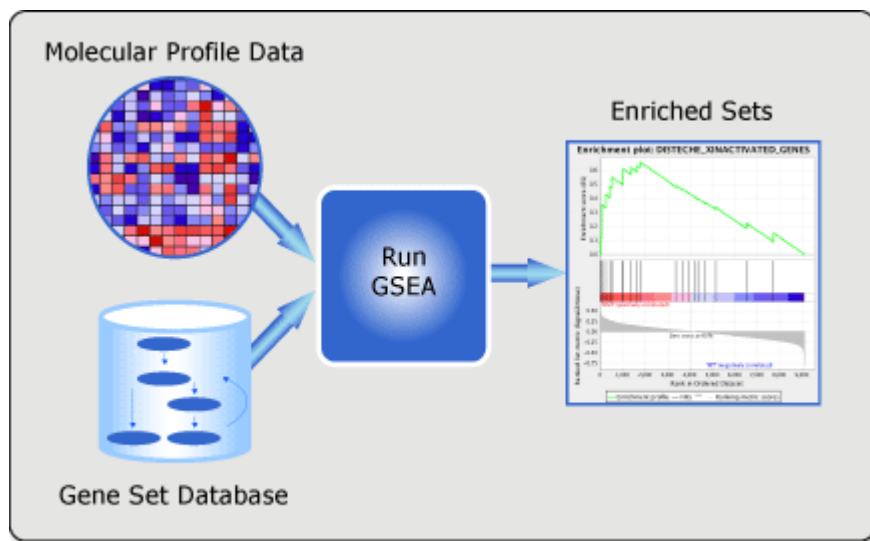
Which cluster relates to Lymphatic cells? And to EPCs?

```
celltypes = {'0': '0', '1': 'Stalk-like cells', '2': 'Lymphatic', '3': '3', '4': '4', '5': 'T
```

```
adata.obs['celltypes'] = list(adata.obs['louvain2'])
adata.obs['celltypes'] = adata.obs['celltypes'].map(celltypes)
```

```
adata.obs['celltypes'] = adata.obs['celltypes'].astype('category')
```

GSEA with KEGG database



```
!pip install gseapy
```

Collecting gseapy

Downloading gseapy-1.1.3-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
552.9/552.9 kB 7.9 MB/s eta 0:00:00

Requirement already satisfied: numpy>=1.13.0 in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: matplotlib>=2.2 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: cyclery>=0.10 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from gseapy)
Installing collected packages: gseapy

Successfully installed gseapy-1.1.3

```
import gseapy as gp
```

```
# gseapy expects as input a matrix in pandas dataframe format and a list with the related
matrix = adata[(adata.obs['louvain2']=='0')| (adata.obs['louvain2']=='3')].to_df().T
celltypes = adata[(adata.obs['louvain2']=='0')| (adata.obs['louvain2']=='3')].obs.louvain2
```

```
import gc
gc.collect()
```

18053

```
if not os.path.exists('kegg_pathways.gmt'):
    !wget --no-check-certificate 'https://docs.google.com/uc?export=download&id=1J5ZMGoRuC
--2024-05-22 10:25:59--  https://docs.google.com/uc?export=download&id=1J5ZMGoRuCQct7Tm
Resolving docs.google.com (docs.google.com)... 108.177.11.113, 108.177.11.139, 108.177
Connecting to docs.google.com (docs.google.com)|108.177.11.113|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=1J5ZMGoRuCQct7TmN07MaVZC_nH
--2024-05-22 10:26:00--  https://drive.usercontent.google.com/download?id=1J5ZMGoRuCQc
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 108.177.12.13
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|108.177.12.1
HTTP request sent, awaiting response... 200 OK
Length: 84217 (82K) [application/octet-stream]
Saving to: 'kegg_pathways.gmt'

kegg_pathways.gmt  100%[=====>]  82.24K  --.-KB/s    in 0.001s

2024-05-22 10:26:00 (101 MB/s) - 'kegg_pathways.gmt' saved [84217/84217]
```

```
res_kegg = gp.gsea(data = matrix,
                    gene_sets='./kegg_pathways.gmt',
                    cls=list(celltypes), # Path to the class labels file indicating the phe
                    permutation_num=100,
                    permutation_type='phenotype',
                    outdir=None,
                    method='s2n',
                    threads=4)
```

```
res_kegg.res2d.sort_values('NES', ascending = False)
```

	Name	Term	ES	NES	NOM p- val	FDR q va
0	gsea	KEGG_RIBOSOME	0.965942	2.718438	0.0	0.
1	gsea	KEGG_CARDIAC_MUSCLE_CONTRACTION	0.673551	2.640162	0.0	0.
2	gsea	KEGG_ADHERENS_JUNCTION	0.561405	2.543119	0.0	0.
3	gsea	KEGG_HUNTINGTONS_DISEASE	0.514862	2.490964	0.0	0.
4	gsea	KEGG_OXIDATIVE_PHOSPHORYLATION	0.548549	2.462357	0.0	0.

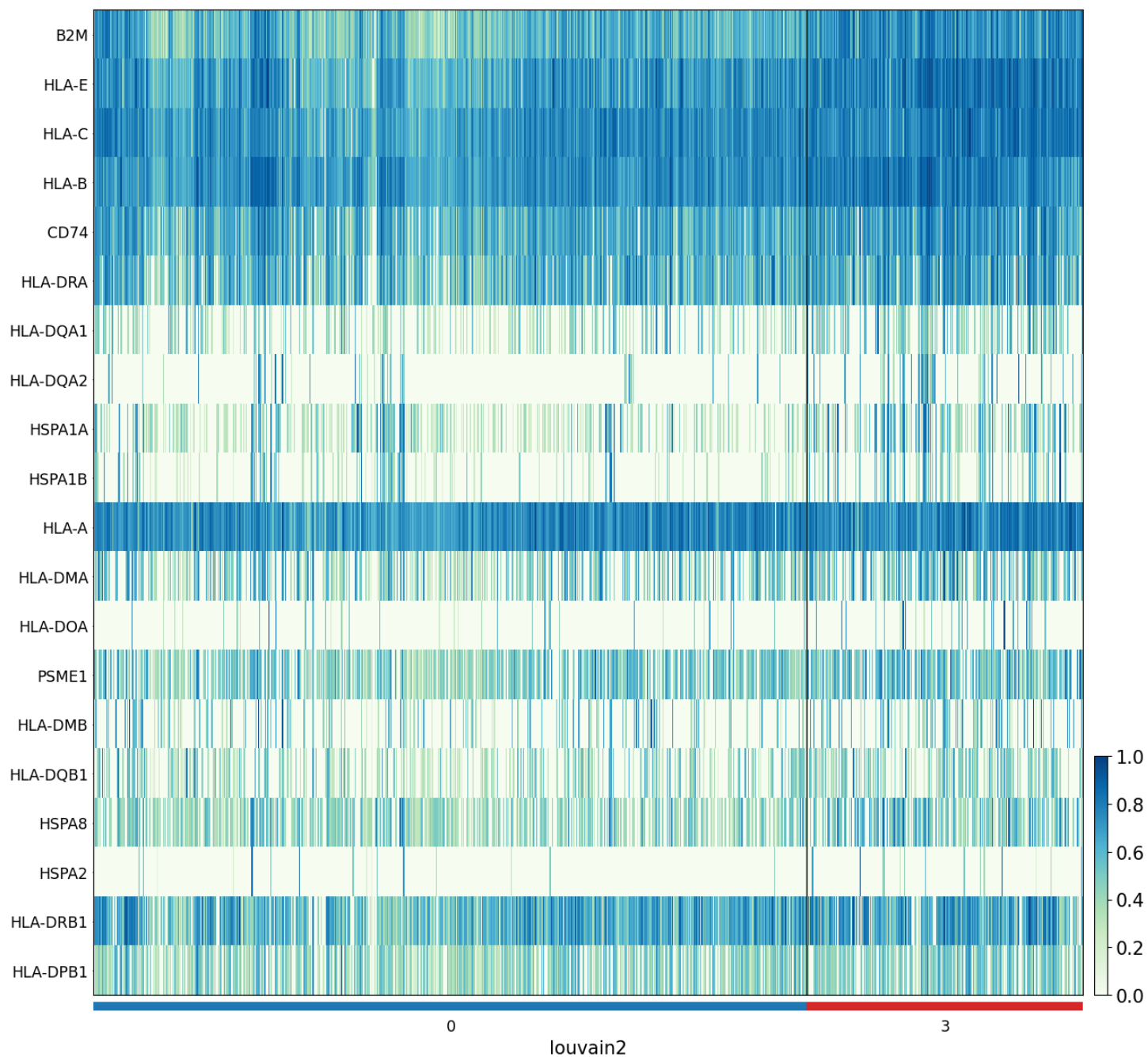
```
res_kegg.res2d = res_kegg.res2d[res_kegg.res2d['FWER p-val'] < 0.05].sort_values('NES', as
```

```
res_kegg.res2d
```

	Name	Term	ES	NES	NOM p- val	FDR
0	gsea	KEGG_RIBOSOME	0.965942	2.718438	0.0	
1	gsea	KEGG_CARDIAC_MUSCLE_CONTRACTION	0.673551	2.640162	0.0	
2	gsea	KEGG_ADHERENS_JUNCTION	0.561405	2.543119	0.0	
3	gsea	KEGG_HUNTINGTONS_DISEASE	0.514862	2.490964	0.0	
4	gsea	KEGG_OXIDATIVE_PHOSPHORYLATION	0.548549	2.462357	0.0	
5	gsea	KEGG_PARKINSONS_DISEASE	0.54928	2.353572	0.0	
6	gsea	KEGG_RENAL_CELL_CARCINOMA	0.486155	2.310851	0.0	
8	gsea	KEGG_ALZHEIMERS_DISEASE	0.474745	2.281062	0.0	
12	gsea	KEGG_PANCREATIC_CANCER	0.484789	2.192933	0.0	
13	gsea	KEGG_PATHWAYS_IN_CANCER	0.412649	2.179717	0.0	
15	gsea	KEGG_JAK_STAT_SIGNALING_PATHWAY	0.43778	2.12576	0.0	0.001
16	gsea	KEGG_FOCAL_ADHESION	0.429212	2.120527	0.0	0.001
17	gsea	KEGG_COMPLEMENT_AND_COAGULATION_CASCADES	0.558224	2.119242	0.0	0.001
18	gsea	KEGG_SPLICEOSOME	0.42616	2.111349	0.0	0.001
19	gsea	KEGG_PROTEIN_EXPORT	0.660502	2.082331	0.0	0.000
20	gsea	KEGG_REGULATION_OF_ACTIN_CYTOSKELETON	0.400702	2.053897	0.0	0.000
21	gsea	KEGG_ACUTE_MYELOID_LEUKEMIA	0.485384	2.01698	0.0	0.002
22	gsea	KEGG_INSULIN_SIGNALING_PATHWAY	0.407201	2.005126	0.0	0.002
23	gsea	KEGG_CHRONIC_MYELOID_LEUKEMIA	0.440353	1.998634	0.0	0.002
24	gsea	KEGG_THYROID_CANCER	0.558702	1.997741	0.0	0.002
14	gsea	KEGG_AUTOIMMUNE_THYROID_DISEASE	-0.783371	-2.178478	0.0	0.00
11	gsea	KEGG_ALLOGRAFT_REJECTION	-0.740441	-2.200795	0.0	0.001
10	gsea	KEGG_TYPE_I_DIABETES_MELLITUS	-0.703278	-2.244466	0.0	
9	gsea	KEGG_ANTIGEN_PROCESSING_AND_PRESENTATION	-0.629388	-2.260377	0.0	
7	gsea	KEGG_GRAFT_VERSUS_HOST_DISEASE	-0.749451	-2.307005	0.0	

```
import matplotlib.pyplot as plt
plt.rcParams['image.cmap'] = 'GnBu'
sc.pl.heatmap(adata[(adata.obs.leuven2 == '0') | (adata.obs.leuven2 == '1')])
```

```
sc.pl.heatmap(adata[(adata.obs.louvain2 == 0) | (adata.obs.louvain2 == 3)],
              list(res_kegg.res2d[res_kegg.res2d.Term.str.contains('KEGG_ANTIGEN_PROCESSIN
groupby='louvain2', standard_scale='var', vmax=1, swap_axes=True,
figsize=[15,15], show_gene_labels=True)
```



GSEA with GO database to infer differences between clusters

```
if not os.path.exists('go_pathways.gmt'):
    !wget --no-check-certificate 'https://docs.google.com/uc?export=download&id=1TaTPb9Kp6
    --2024-05-22 10:26:45-- https://docs.google.com/uc?export=download&id=1TaTPb9Kp6qFJPI
    Resolving docs.google.com (docs.google.com)... 108.177.11.102, 108.177.11.139, 108.177
    Connecting to docs.google.com (docs.google.com)|108.177.11.102|:443... connected.
    HTTP request sent, awaiting response... 303 See Other
    Location: https://drive.usercontent.google.com/download?id=1TaTPb9Kp6qFJPIJI_mtECNex1D
    --2024-05-22 10:26:45-- https://drive.usercontent.google.com/download?id=1TaTPb9Kp6qF
    Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 108.177.12.13
    Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|108.177.12.1
    HTTP request sent, awaiting response... 200 OK
    Length: 1380288 (1.3M) [application/octet-stream]
    Saving to: 'go_pathways.gmt'

go_pathways.gmt      100%[=====>]    1.32M  --.-KB/s    in 0.009s

2024-05-22 10:26:46 (142 MB/s) - 'go_pathways.gmt' saved [1380288/1380288]
```

```
res_go = gp.gsea(data = matrix,
                 gene_sets='./go_pathways.gmt',
                 cls=list(celltypes),
                 permutation_num=100,
                 permutation_type='phenotype',
                 outdir=None,
                 method='s2n',
                 threads=4)
```

```
res_go.res2d.sort_values('NES', ascending = False)
```

	Name	Term	ES	NES	NOM p- val	FDR q- val	FWER p- val	Tag %	Gene %	
0	gsea	Gene Expression (GO:0010467)	0.730521	3.4963	0.0	0.0	0.0	81/273	2.09%	RPS6;RP
1	gsea	Translation (GO:0006412)	0.779817	3.353739	0.0	0.0	0.0	88/211	5.72%	RPS6;RP
2	gsea	Macromolecule Biosynthetic Process (GO:0009059)	0.82235	3.353545	0.0	0.0	0.0	90/166	7.20%	RPS6;RP
3	gsea	Peptide Biosynthetic Process (GO:0043043)	0.859384	3.238456	0.0	0.0	0.0	76/140	2.07%	RPS6;RP
4	gsea	Ribonucleoprotein Complex Biogenesis (GO:0022613)	0.719039	3.026843	0.0	0.0	0.0	48/113	12.04%	RPS6;RP

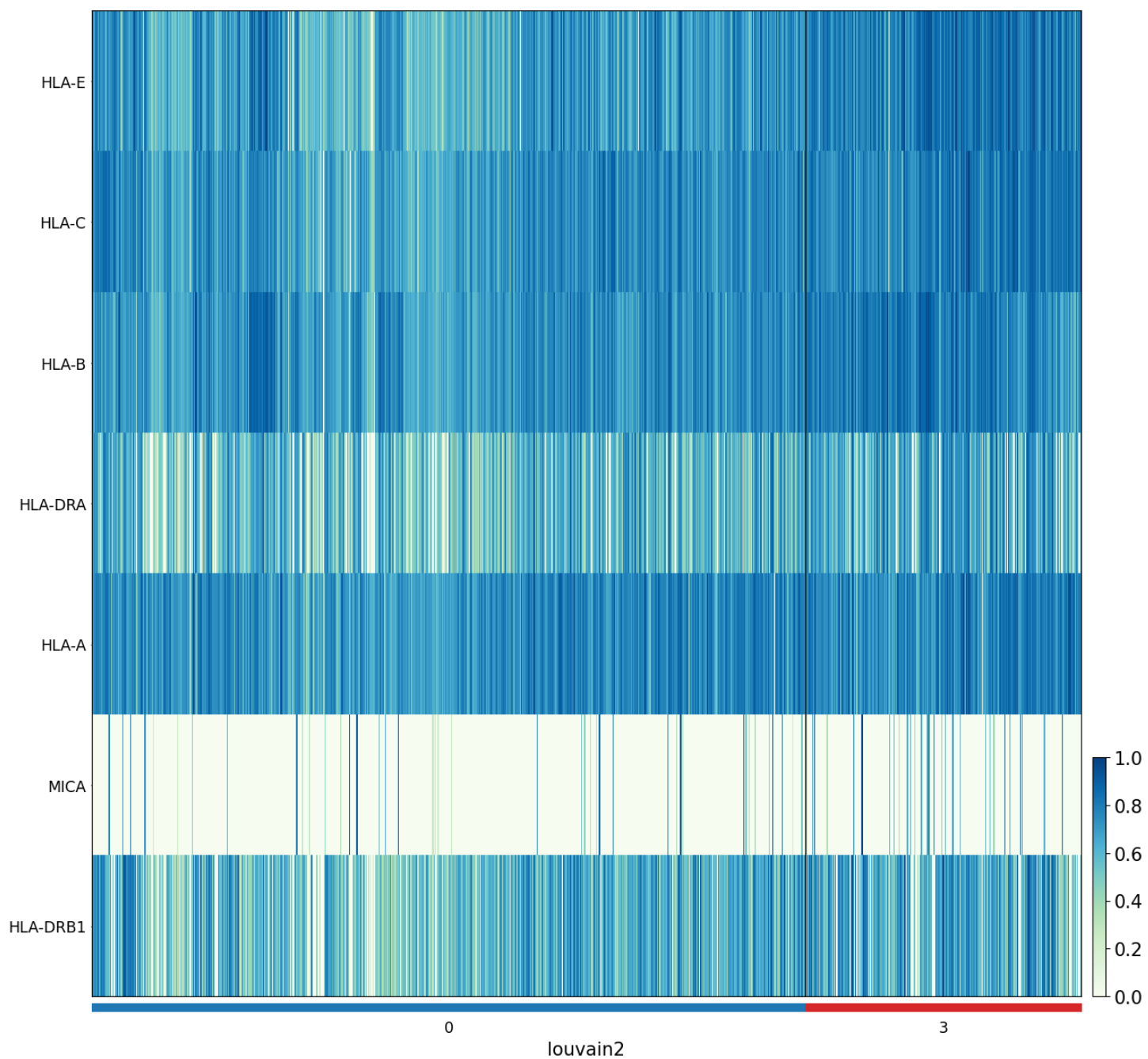
```
res_go.res2d = res_go.res2d[res_go.res2d['FWER p-val'] < 0.05].sort_values('NES', ascendin
```

res_go.res2d.tail(30)

	Name	Term	ES	NES	NOM p- val	FDR q- val	FWER p- val	Tag %	Gene %	
63	gsea	Negative Regulation Of Multicellular Organisma...	0.419718	2.247068	0.0	0.001451	0.02	74/184	25.28%	
64	gsea	Positive Regulation Of Leukocyte Migration (GO...	0.658483	2.24594	0.0	0.001427	0.02	7/24	5.73%	
65	gsea	Positive Regulation Of Macromolecule Metabolic...	0.420096	2.245758	0.0	0.001404	0.02	101/264	24.29%	
66	gsea	RNA Splicing, Via Transesterification Reaction...	0.413132	2.241431	0.0	0.001381	0.02	78/170	35.58%	HM
67	gsea	Positive Regulation Of Cell Migration (GO:0030...	0.421738	2.235184	0.0	0.001359	0.02	99/230	28.38%	
68	gsea	Platelet Aggregation (GO:0070527)	0.622368	2.23199	0.0	0.001561	0.03	12/29	15.74%	
69	gsea	Epithelial Tube Morphogenesis (GO:0060562)	0.616126	2.229511	0.0	0.001537	0.03	10/27	10.37%	
70	gsea	Bone Development (GO:0060348)	0.605253	2.228933	0.0	0.001513	0.03	8/32	7.86%	
71	gsea	Positive Regulation Of Cell Motility (GO:2000147)	0.446523	2.223655	0.0	0.001491	0.03	85/184	28.38%	
72	gsea	Heart Trabecula Morphogenesis (GO:0061384)	0.682451	2.220374	0.0	0.001469	0.03	6/15	10.80%	
73	gsea	U2-type Prespliceosome Assembly (GO:1903241)	0.661221	2.218493	0.0	0.001448	0.03	14/23	18.87%	S
74	gsea	Positive Regulation Of Cell Adhesion (GO:0045785)	0.533232	2.21742	0.0	0.001427	0.03	31/69	22.14%	-

```
import matplotlib.pyplot as plt
plt.rcParams['image.cmap'] = 'GnBu'
sc.pl.heatmap(adata[(adata.obs.louvain2 == '0') | (adata.obs.louvain2 == '3')],
              list(res_go.res2d[res_go.res2d.Term.str.contains('Antigen Processing And Pre
groupby='louvain2', standard_scale='var', vmax=1, swap_axes=True,
```

```
figsize=[15,15], show_gene_labels=True)
```

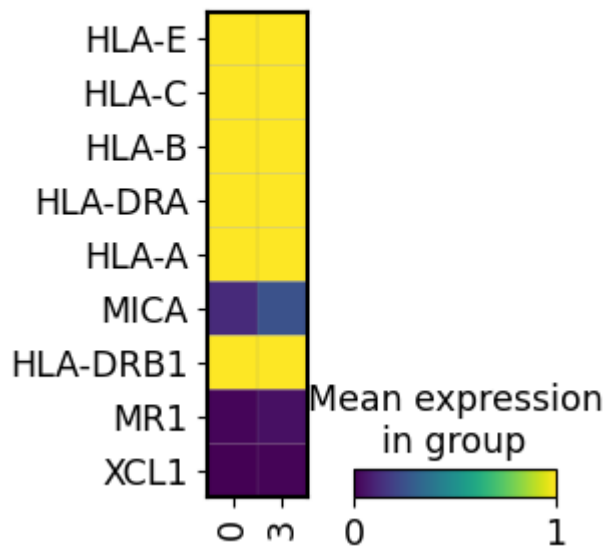
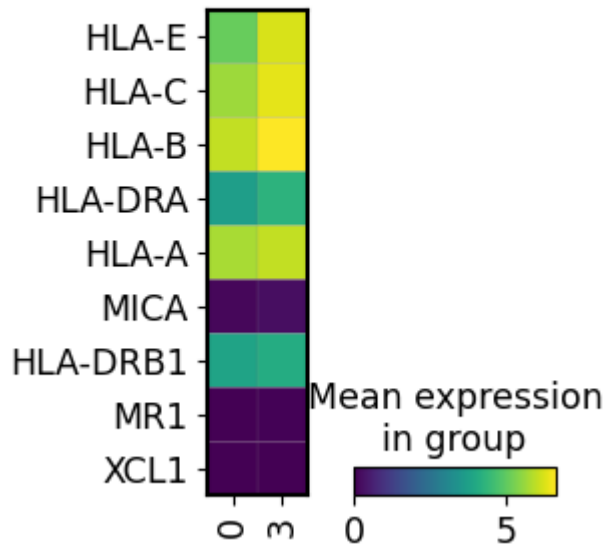


```
import matplotlib.pyplot as plt
plt.rcParams['image.cmap'] = 'GnBu'
```

```
sc.pl.matrixplot(adata[(adata.obs.louvain2 == '0') | (adata.obs.louvain2 == '3')],
```

```
list(res_go.res2d[res_go.res2d.Term.str.contains('Positive Regulation Of
groupby='louvain2', swap_axes=True)
```

```
sc.pl.matrixplot(adata[(adata.obs.louvain2 == '0') | (adata.obs.louvain2 == '3')],
list(res_go.res2d[res_go.res2d.Term.str.contains('Positive Regulation Of
groupby='louvain2', swap_axes=True, vmax = 1)
```



```
import matplotlib.pyplot as plt
plt.rcParams['image.cmap'] = 'GnBu'

sc.pl.matrixplot(adata[(adata.obs.louvain2 == '0') | (adata.obs.louvain2 == '3')],
list(res_go.res2d[res_go.res2d.Term.str.contains('0001570')].Lead_genes.s
groupby='louvain2', swap_axes=True)

sc.pl.matrixplot(adata[(adata.obs.louvain2 == '0') | (adata.obs.louvain2 == '3')],
list(res_go.res2d[res_go.res2d.Term.str.contains('0001570')].Lead_genes.s
groupby='louvain2', swap_axes=True, vmax = 1)
```

IndexError Traceback (most recent call last)

```

3
4 sc.pl.matrixplot(adata[(adata.obs.louvain2 == '0') | (adata.obs.louvain2 ==
'3')],
----> 5
list(res_go.res2d[res_go.res2d.Term.str.contains('0001570')].Lead_genes.str.split(';')
[0],
6
7
groupby='louvain2', swap_axes=True)

Today's Error: list index out of range

```

GSEA to infer differences between Conditions

```
adata.obs
```

```

matrix = adata[(adata.obs['origin']=='nLung') | (adata.obs['origin']=='tLung')].to_df().T
celltypes = adata[(adata.obs['origin']=='nLung') | (adata.obs['origin']=='tLung')].obs.orig

```

```

res_go = gp.gsea(data = matrix,
                 gene_sets='./go_pathways.gmt',
                 cls=list(celltypes),
                 permutation_num=100,
                 permutation_type='phenotype',
                 outdir=None,
                 method='s2n',
                 threads=4)

```

```
res_go.res2d = res_go.res2d[res_go.res2d['FWER p-val'] < 0.05].sort_values('NES', ascending=False)
```

```
res_go.res2d.tail(30)
```

```

import matplotlib.pyplot as plt
plt.rcParams['image.cmap'] = 'GnBu'

sc.pl.matrixplot(adata[(adata.obs['origin']=='nLung') | (adata.obs['origin']=='tLung')],
                 list(res_go.res2d[res_go.res2d.Term.str.contains('0050853')].Lead_genes.str.split(';')
groupby='origin', swap_axes=True)

sc.pl.matrixplot(adata[(adata.obs['origin']=='nLung') | (adata.obs['origin']=='tLung')],
                 list(res_go.res2d[res_go.res2d.Term.str.contains('0050853')].Lead_genes.str.split(';')
groupby='origin', swap_axes=True, vmax = 1)

```